Szent János Görögkatolikus Gimnázium, Szakgimnázium, Technikum, Kollégium és Óvoda

DÖKNet

Témavezető:
Kovács Donát
Rendszergazda

Készítette:
Krajnyák Máté
Tóth Dominik
Kévés Bálint

Edelény

2026

Tartalomjegyzék

Bevezetés **4**

Projektmunka témája és funkciói 4
Miért DÖKNet? 4
Kiknek szánjuk ezt a programot? 4
Miben más, mint a többi? 4

1. Fejlesztői dokumentáció **5**

1.1. A projekt elkészítéséhez használt szoftverek 5
1.2. A projekthez használt programnyelvek 8

2. A projekt megtervezése **10**

3. Az oldal Fő funkciói **11**

3.1. Főoldal 11
3.1.1. Képnézegető 11
3.1.2. Hírek 11
3.2. Naptár 12
3.3. Galéria 13
3.4. Hírszerkesztő 14
3.4.1. A szerkesztő 14
3.4.2. Az elkészült hír 15
3.4.3. Létező hír szerkesztése 16
3.5. Szavazás 18
3.5.1. Listázott szavazások 18
3.5.2. Szavazások létrehozása 18
3.5.3. Elkészült szavazás 19
3.5.4. Lezárt szavazás 20
3.6. DÖKChat 22
3.7. Admin panel 23

4. Regisztráció/Bejelentkezés **24**

4.1. Regisztráció 25
4.2. Bejelentkezés 26

5. A DÖKNet adatbázisa **27**

5.1. users 28
5.2. news 29
5.3. reaction_types 30
5.4. news_reactions 31
5.5. comments 32
5.6. chat_messages 33
5.7. gallery 34
5.8. events 35
5.9. votes 36
5.10. vote_options 37
5.11. vote_votes 38

6. A weblap designja **39**

6.1. Logó 39
6.2. Szín paletta 40
6.3. Gombok, inputok 41
6.4. Reszponzív kialakítás 41
6.5. Navigáció és felhasználói élmény 42

7. Frontend ... 43
7.1. Technológiai alapok ... 43
7.2. Komponens alapú felépítés ... 44
7.3. Állapotkezelés ... 45
8. Backend ... 46
8.1. Backend architektúra és működési modell ... 46
8.2. API réteg és végpontok részletes működése ... 47
8.2.1. Bevezetés ... 47
8.2.2. A projekt API végpontjai csoportosítva ... 47
8.2.3. Működési logika ... 48
8.2.4. Kapcsolat a frontenddel ... 48
8.3. Adatkezelés és adatbázis műveletek ... 49
8.4. Hitelesítés és jogosultságkezelés ... 50
8.5. Hibakezelés, validáció és rendszerstabilitás ... 51
9. Tesztelés ... 52
9.1. Menete ... 52
9.2. Tapasztalatok ... 52
9.3. Konklúzió ... 52
10. Későbbi fejlesztési lehetőségek ... 53
10.1. Chat ... 53
10.2. Naptár ... 53
10.3. Galéria ... 53
10.4. Főoldal ... 54
10.5. Szavazás ... 54
10.6. Hírek ... 54
10.7. Dokumentumtár ... 54
10.8. Közösségi szolgálat követő ... 54
10.9. Értesítéskezelés ... 55
10.10. QR-kód generátor ... 55
10.11. „Digitális Szekrény” ... 55
10.12. „Öreg Diák Fal” ... 55
10.13. Profil beállítások ... 55
Összefoglalás ... 56
Summary ... 57
Ábrajegyzék ... 58
Hivatkozások ... 60

Bevezetés

Projektmunka témája és funkciói

A projekt célja, hogy megkönnyítse a diákönkormányzati tagok dolgát az események lebonyolításában. Az oldal segíti a dök tagok kommunikációját, események naptárban való feljegyzését, eldöntendő kérdések megszavazását továbbá hírek írására és olvasására is lehetőséget kínál, valamint a galéria menüpont használatával lehetőséget kínálunk a diákönkormányzathoz kapcsolódó eseményekről készült képek megtekintésére.

Miért DÖKNet?

A DÖKNet nevű projektünk segíteni szeretne az iskolánk diákönkormányzatának. Az oldalunk egy egyszerű és felhasználóbarát megoldáson keresztül szeretné segíteni a dök működését.

Kiknek szánjuk ezt a programot?

Azért hoztuk létre a programunkat mert az iskolának eddig még nem volt ilyen célra szánt webes oldala. Emiatt projektünket leginkább diákönkormányzat tagoknak és az ő munkájukat segítő tanároknak ajánljuk. Az oldalt úgy készítettük, hogy átlagos diákok számára is érdekes lehessen, a főoldalon megjelenő hírek és a galéria funkció segítségével.

Miben más, mint a többi?

Utána járásunkat követően megállapítottuk, hogy a DÖKNet-hez hasonló oldalak eddig nem igazán voltak, vagy amik voltak azok is régiek és elavultak voltak. Ebből fakadóan a mi oldalunk modernebb szebb és praktikusabb a felhasználást illetően. Ezzel egy a diákok számára is kedvelhető és szívesen használható oldalt tudtunk létrehozni. A DÖKNet-hez hasonló oldal hiányában teljesen a mi képzeletünkre és szándékunkra volt bízva az oldal tartalmi és funkcionális része is.

1. Fejlesztői dokumentáció

1.1. A projekt elkészítéséhez használt szoftverek

1. ábra: Visual Studio Code logó

Visual Studio Code

A Visual Studio Code egy ingyenes, nyílt forráskódú forráskód-szerkesztő, amelyet a Microsoft fejlesztett ki.
(Forrás: https://www.prooktatas.hu/hirek/visual-studio-code-hasznalata)

A VS Code számos bővítményével segítette a munkánkat, ilyen például a `Prettier`. A projekt készítése alatt végig ezt a kódszerkesztőt használtuk.

2. ábra: XAMPP logó

XAMPP

A `XAMPP` egy szabad és nyílt forrású platformfüggetlen webszerver-szoftvercsomag, amelynek legfőbb alkotóelemei az Apache webszerver, a MariaDB adatbázis-kezelő, valamint a PHP és a Perl programozási nyelvek értelmezői. *(Forrás: https://hu.wikipedia.org/wiki/XAMPP)*

A `XAMPP`-ban dolgozni egyszerű volt és könnyű. Pár kattintással futtatni tudtuk a webszerverünket és az adatbázisunkat. A `phpMyAdmin` nagyban megkönnyítette a dolgunkat az adatbázis tervezése és készítése közben.

3. ábra: Node.js logó

Node.js

A Node.js egy nyílt forráskódú, platformfüggetlen JavaScript futtatókörnyezet. Ez egy népszerű eszköz majdnem minden fajta projekthez!
(Forrás: https://nodejs.org/en/learn/getting-started/introduction-to-nodejs)

`Node.js`-vel dolgozni azért volt számunkra könnyebbség, mert ennek használatával nem kellett másik programozási nyelvet használnunk a projekt `backend`jének készítése közben. Az `npm` csomagok használata egyszerű és sokkal könnyebbé tette számunkra a munkát.

4. ábra: ChatGPT logó

ChatGPT

Néhány bonyolultabb feladat megvalósításában segített számunkra az egyik legnépszerűbb nagy nyelvi modell az OpenAI ChatGPT-je, valamint egyes projekt részek elméleti kidolgozásában alkalmaztuk. Továbbá még hibák keresésére használtuk. Az MI-tól kapott válaszokat minden esetben átnéztük, hogy nehogy valamit hibásan illesszünk be a programunkba.
(A ChatGPT elérhető itt: https://chatgpt.com)

5. ábra: Google Gemini

Google Gemini

A `Google Gemini` egy generatív mesterséges intelligencia (MI) alapú chatbot, amelyet a Google fejlesztett ki.

(Forrás: https://hu.wikipedia.org/wiki/Gemini_(chatbot))

A logónk tervezéséhez használtuk ezt az MI-t. A részletes leírásunk után generáltattuk le. A leírásban a következőket adtuk meg: főként bordó színek, modern stílusban, bagoly fej, „DÖKNet” felirat az alján, néhány szimbólum, ami a rendezvényekre és az oktatásra utal.

(A Gemini elérhető itt: https://gemini.google.com/app)

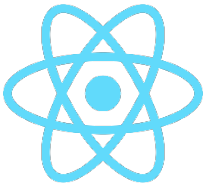
1.2. A projekthez használt programnyelvek

6. ábra: Next.js logó

Next.js

A Next.js egy React-re épülő keretrendszer full-stack webalkalmazások készítéséhez. A felhasználói felületet React komponensek segítségével építhetjük fel, a Next.js azonban további funkciókat és optimalizációkat ad hozzá.
(Forrás: https://nextjs.org/docs#what-is-nextjs)

A Next.js használata sokban megkönnyítette a dolgunkat azzal, hogy a backend és a frontend részt is egy projekten belül tudtuk fejleszteni.



7. ábra: React logó

React

A React egy ingyenes nyílt forráskódú frontend JavaScript könyvtár, ami arra épít, hogy felhasználói felületeket készítünk komponensekkel. A React használható web és mobil appok fejlesztésére is.
(Forrás: https://en.wikipedia.org/wiki/React_(software))

React-val dolgozni könnyű volt és gyorsan tanulható. A komponensek használata sokban egyszerűsítette a dolgunkat, azzal, hogy ugyanazt a program részt nem kellett többször is megírnunk.

8. ábra: TypeScript logó

TypeScript

A TypeScript egy magas szintű programozási nyelv, ami statikus típusosságot ad opcionális típusmegjelöléssel a JavaScriptnek. Nagyobb méretű alkalmazások fejlesztésére tervezték. A kódot JavaScriptre fordítja.
(Forrás: https://en.wikipedia.org/wiki/TypeScript)

A TypeScript-et a DÖKNet backendjének fejlesztésére használtuk. Ez a nyelv átláthatóbbá tette a kód írást. A szintaxisa mivel nagyban hasonlít a JavaScriptére ezért jelentősebb probléma nélkül tanulhattuk meg.

9. ábra: MySQL logó

MySQL

A MySQL egy nyílt forráskódú relációs adatbázis kezelő rendszer, ami adatok tárolására és kezelésére szolgál. Megbízhatósága, teljesítménye, skálázhatósága és könnyű használata miatt tette népszerűvé a MySQL-t a fejlesztők körében.
(Forrás: https://www.oracle.com/mysql/what-is-mysql/)

A projekt adatainak tárolása és kezelése MySQL-ben történt, a phpMyAdmin sokkal gyorsabbá és egyszerűbbé tette az adatok ellenőrzését.

2. A projekt megtervezése

A DÖKNet rendszer fejlesztésének során az elsődleges célunk egy olyan iskolai DÖK platform létrehozása volt, amivel egyszerűen használható, modern kinézetű és egy helyen biztosító több, a diákok számára hasznos funkciót biztosítunk. A projekt fejlesztése nem egy teljesen előre megtervezett struktúra alapján történt, hanem fokozatosan, iteratív módon alakult ki a fejlesztés közben.

A munka kezdetén először a főoldal és a hírszerkesztő rendszer készültek el, mivel ezek képezték az alkalmazás alapját. A későbbi funkciókat és rendszerkomponenseket fejlesztés közben terveztük meg és alakítottuk ki. A projekt tervezése főként a csapattagok közötti egyeztetésekre épült, külön tervezőprogram vagy diagramok használata nélkül. Az egyes ötleteket és funkciókat a fejlesztés során folyamatosan átbeszéltük, majd ezek alapján alakítottuk ki a rendszer működését és felépítését.

A projekt során több funkció is menet közben került be a rendszerbe. Ilyen volt például a galéria és az admin panel, amelyek eredetileg nem szerepeltek a kezdeti elképzelések között, azonban a fejlesztés során felmerült igények miatt végül hozzáadtuk a rendszerhez. Emiatt az adatbázis szerkezete sem egyszerre készült el, hanem folyamatosan bővült az új funkciók igényeinek megfelelően.

A fejlesztés egyik legnagyobb kihívását a felhasználóazonosítási és jogosultságkezelési rendszer kialakítása jelentette. A login rendszer működésének megvalósítása és a különböző jogosultsági szintek kezelése több újratervezést és hibajavítást igényelt a fejlesztés során.

A verziókezeléshez GitHubot használtunk, amely lehetővé tette a közös munkát és a projekt változásainak követését. A fejlesztés során a legfontosabb szempont a könnyű használhatóság volt, ezért törekedtünk egy átlátható, egyszerűen kezelhető és modern felhasználói felület kialakítására.

3. Az oldal Fő funkciói

3.1. Főoldal

3.1.1. Képnézegető

A főoldalon található egy interaktív képnézegető, ami a rendezvényekről, iskoláról, fontos eseményekről készült képeket tartalmazza. A képnézegetőben elhelyezett nyilak segítségével manuálisan lapozhatunk a képek között. Reszponzív, tehát mobileszközön vagy táblagépen is remek a kinézete és igazodik a képernyőhöz. A képnézegető a Swiper nevű npm csomag beimportálásával készült el.

10. ábra: A főoldali képnézegető oldalt a logókkal

3.1.2. Hírek

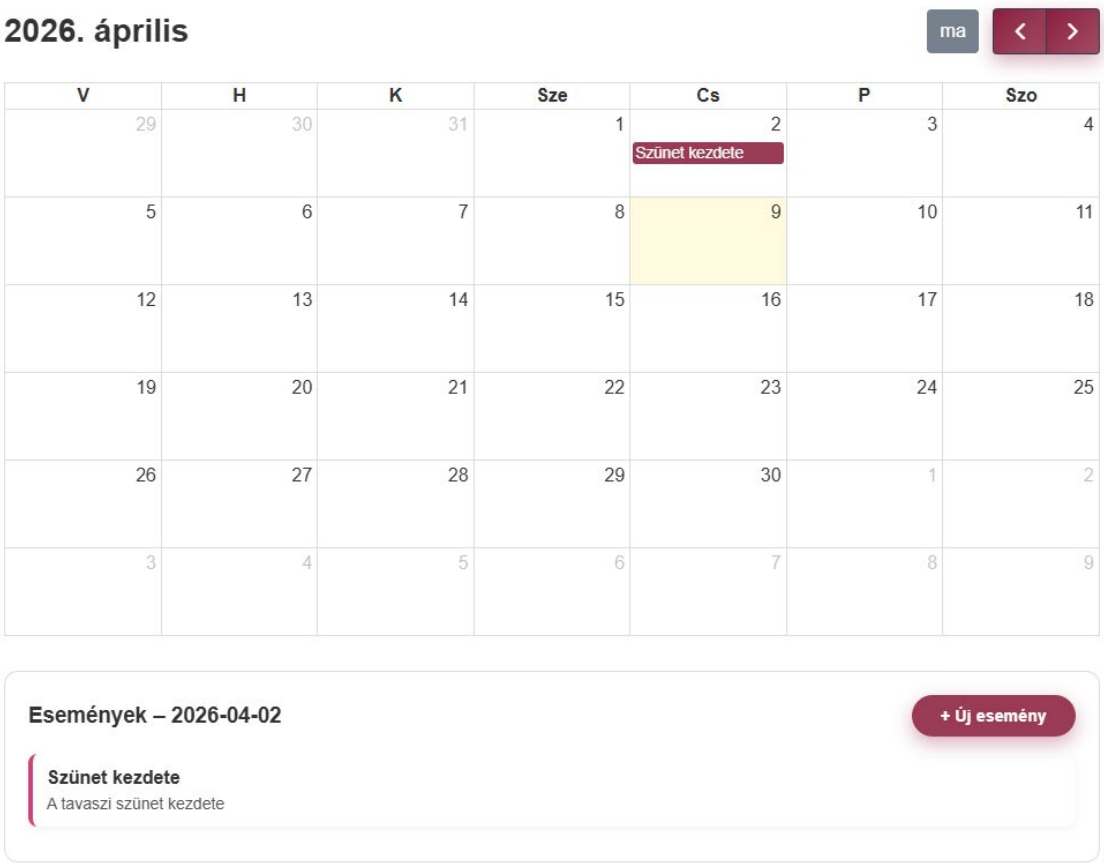
A főoldalon találhatóak a szövegszerkesztőben megírt hírek kilistázva. Az új hír kerül mindig az oldal tetejére. A hírek borítóképe már előre sugallja, hogy az adott hírnek mi lesz a témája. A hírek a főoldalra egy `NewsList.tsx` nevű komponensből vannak be importálva, ami az `api` mappán belüli `news` mappa `route.ts` nevű fájljával kommunikál és kapcsolódik az adatbázishoz.

11. ábra: A főoldali hírek

3.2. Naptár

A naptárt az npm FullCalendar csomagjával hoztuk létre az oldalon. A naptár célja az, hogy az oldalra látogatók megtekinthessék a közeledő eseményeket. Új eseményt létrehozni, szerkeszteni, valamint törölni csak jogosultságokkal meghatározott személyek tudnak. Az imént említett funkciók eléréséhez az adott személynek rendelkeznie kell vagy `Admin` vagy `Tanár` vagy `Elnök` jogosultsággal. A naptárt tekintve ezek a szerepek nem különböznek semmiben egymástól, vagyis mindegyik szerep ugyanazt tudja csinálni a naptár szerkesztési felületén.

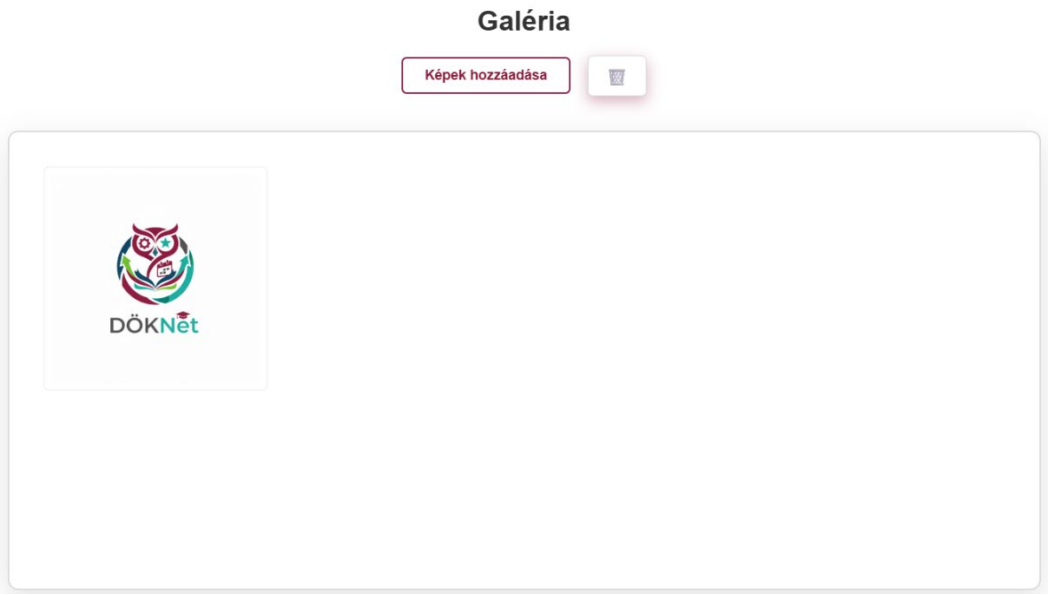
Ha új eseményt hozunk létre, akkor azok az adatbázisba mentődnek el. Új esemény létrehozásánál meg kell adnunk az eseménynek a nevét, és annak leírását. Szerkesztésnél pedig lehetőség van ezeket átírni, és úgy lementeni. Törlésnél ezek az adatok teljesen törlődnek az adatbázisból.



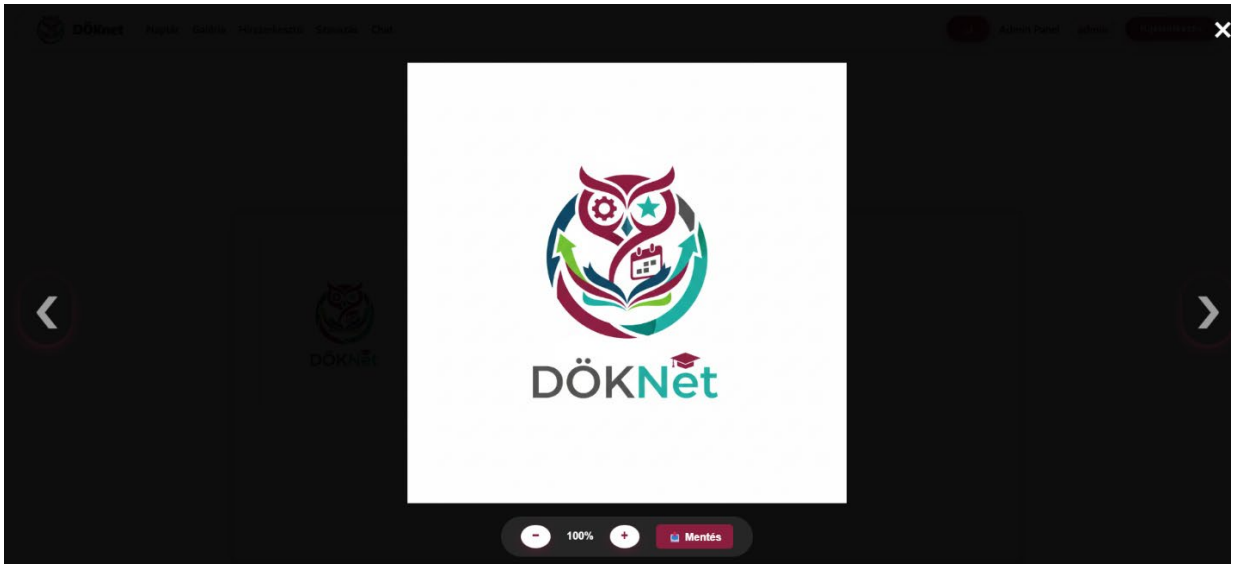
12. ábra: Naptár

3.3. Galéria

Lehetővé teszi, hogy egy nagy adatbázisba feltöltsük az iskoláról készült fotókat. A galériában található képeket tudjuk törölni, akár egyesével vagy ha úgy szeretnénk akkor az összeset. A képekre kattintva megnyitható egy képnézegető, így akár sokkal részletesebben is meg tudjuk nézni a fotókat. A képnézegetőn belül akár csak a főoldalon, el vannak helyezve nyilak a képek váltásához, ezen felül van kép letöltési lehetőség és nagyítási, illetve zsugorítási opció is.



13. ábra: Galéria



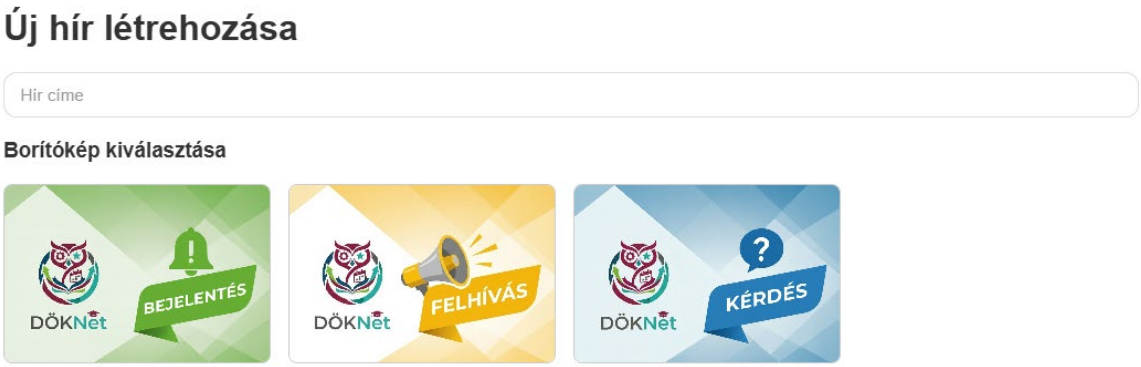
14. ábra: Galéria képnézegetője

3.4. Hírszerkesztő

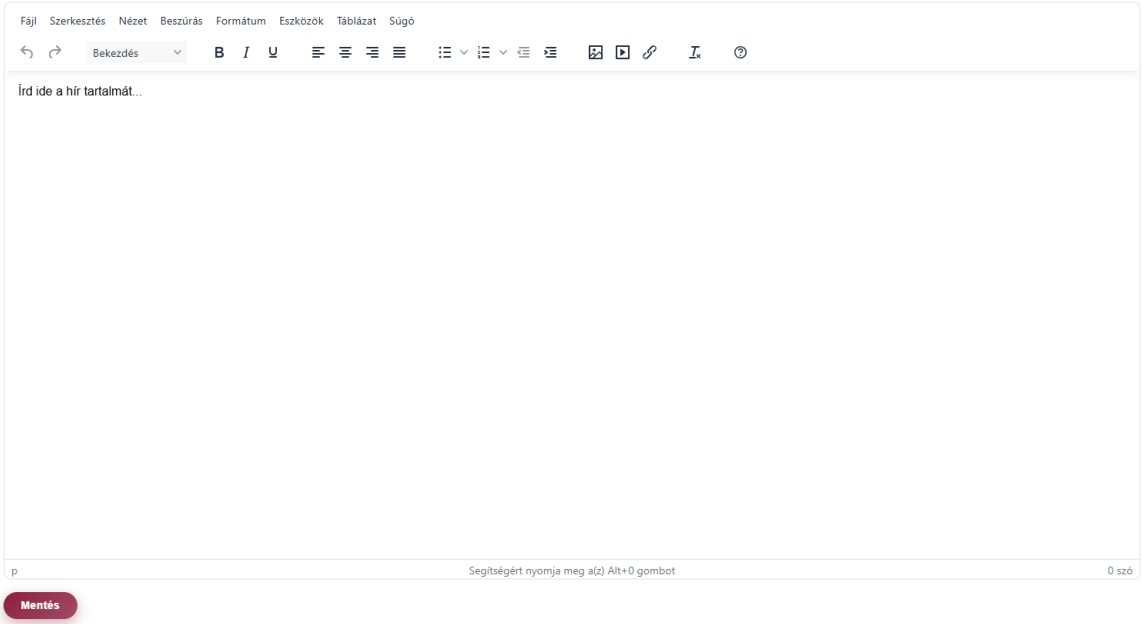
3.4.1. A szerkesztő

Az oldalon a híreket a „Hírszerkesztő” nevű menü pontból lehet létrehozni. A TinyMCE nevű npm csomagot használtuk a szövegszerkesztő megvalósításához, majd ezt bővítettük cím adási lehetőséggel, illetve a 3 előre előkészített borítókép közüli választási lehetőséggel. Ez a funkció csak az `Admin` a `Tanár` vagy az `Elnök` jogosultsággal rendelkező felhasználók számára érhető el.

Az elkészített hírek a mentés gomb megnyomását követően a `news` nevű adatbázis táblában kerülnek elmentésre. Majd ezt követően jelenik meg a főoldalon.



15. ábra: A Hír címe és az előre definiált borítóképek



16. ábra: A szövegszerkesztő mező

3.4.2. Az elkészült hír

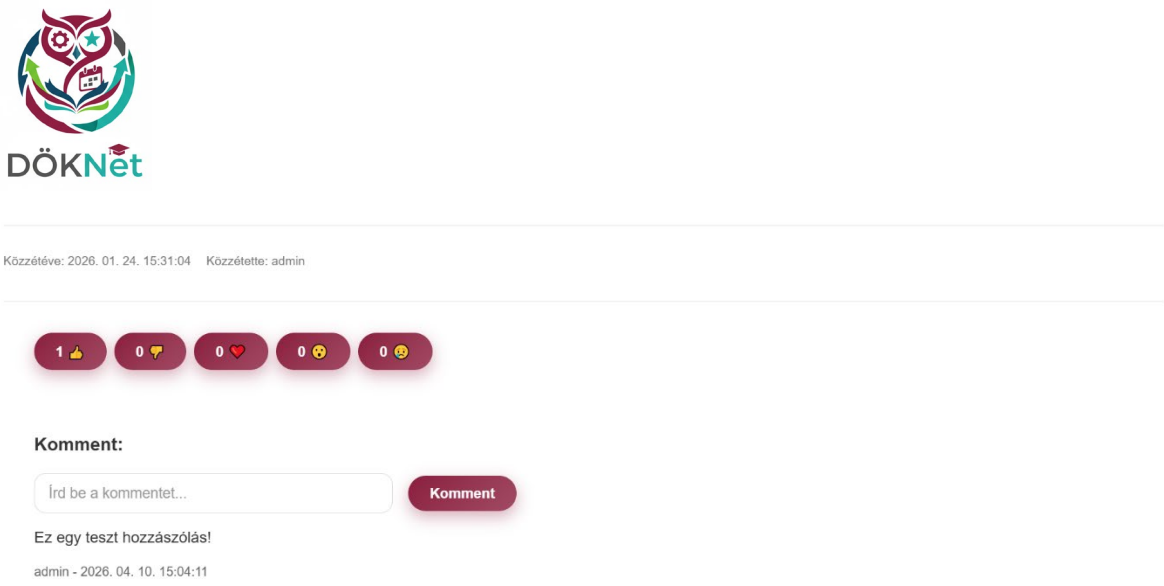
Az elkészült hírek a főoldalról választhatóak ki vagy a hír megírását és mentését követően átirányít a megírt hírre. Itt főként a borítókép a cím és a tartalom látható, azonban az adott jogosultságokkal rendelkező felhasználóknak kettő további gomb válik láthatóvá.

A szerkesztés gomb, ami átirányítja a felhasználót a hírszerkesztő oldalra csak annyi változással, hogy betölti a szerkeszteni kívánt hír adatait. A szerkesztést követően szintén a mentés gomb segítségével lehet véglegesíteni a változtatásokat. A változtatás időpontja és a változtatásokat végző felhasználó `id`-ja szintén mentésre kerül az adatbázisban `modified_at` és `modified_by` néven.

A második gomb a hír törlésének lehetőségét adja az adott jogosultsággal rendelkező felhasználók kezébe. A híreket ténylegesen nem lehet törölni csak a felhasználók számára elérhetetlenné tesszük, „soft deletet” alkalmazunk.

17. ábra: Kész hír eleje

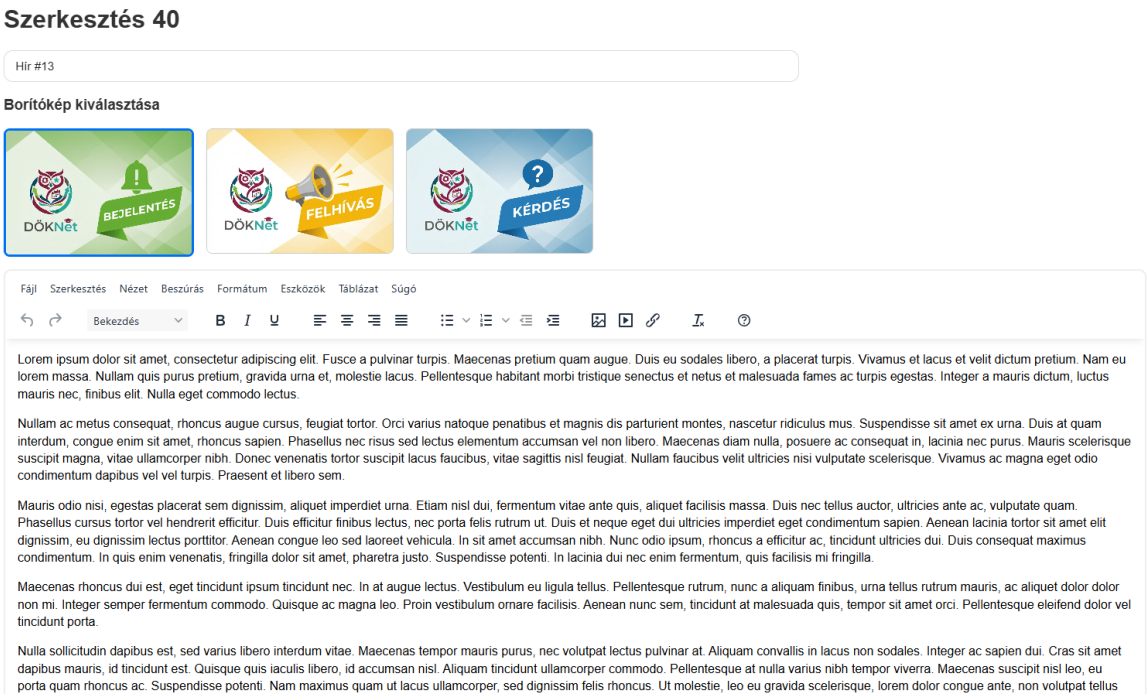
Az oldal alján látható a hír közzétételének pontos dátuma, illetve a hír írójának felhasználó neve. Továbbá öt gomb, amivel reakciót lehet adni az adott hírre és egy komment mező, ahol a felhasználók véleményezhetik vagy hozzá fűzhetnek bármit az adott hírhez. Ezek a funkciók minden fiókkal rendelkező felhasználó számára elérhető.



18. ábra: Kész hír vége

3.4.3. Létező hír szerkesztése

A szerkesztés gomb megnyomását követően az oldal betölti a szerkeszteni kívánt hír adatait a TinyMCE szövegszerkesztőbe. Ezt követően szabadon lehet szerkeszteni a hír címét, botítóképét vagy teljes szövegi tartalmát. A „mentés” gomb megnyomását követően az adatbázis frissíti az adott hírt ért változtatásokat, illetve menti a változtatást végző felhasználó azonosítóját és a változtatások mentésének dátumát.



19. ábra: Kész hír szerkesztése

Közzétéve: 2026. 04. 09. 22:01:39
Közzétette: admin

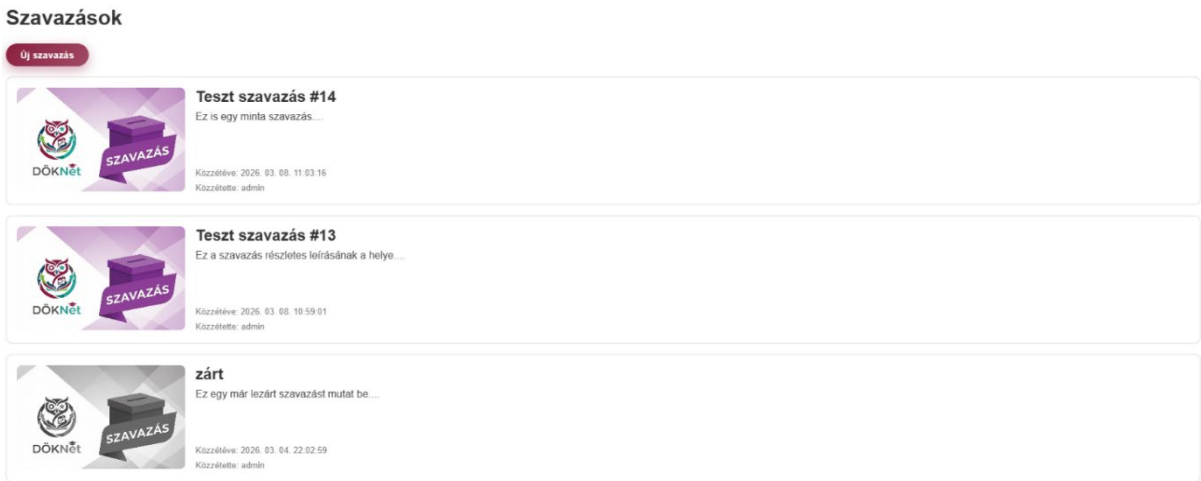
Utoljára módosítva: 2026. 04. 20. 7:32:16
Utoljára módosította: admin

20. ábra: Szerkesztés után

3.5. Szavazás

3.5.1. Listázott szavazások

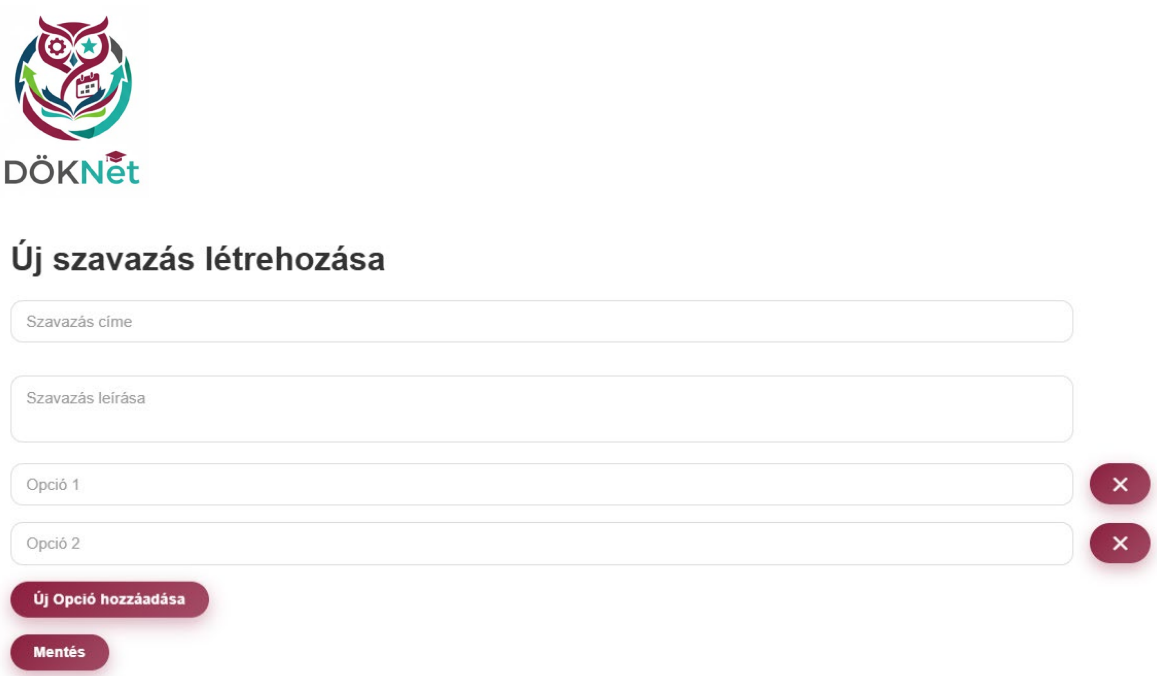
A hírekhez hasonlóan a szavazások is kilistázott formában választhatóak ki, annyi eltéréssel, hogy ezek nem a főoldalon találhatóak meg hanem a „Szavazás” nevű menü pont alatt. Az oldal tetején található egy „Új szavazás” nevű gomb, ami a megfelelő jogosultsággal rendelkező felhasználót (`Tag, Elnök, Tanár, Admin`) átirányítja az „Új szavazás létrehozása” nevű oldal részre. A listánál szintén borítókép, cím és a szavazás leírásából látható néhány részlet. A borítókép színe azt jelenti, hogy amennyiben lila akkor az a szavazás még aktív viszont, ha szürke lesz akkor az már egy lezárt szavazás.



21. ábra: Kilistázott szavazások

3.5.2. Szavazások létrehozása

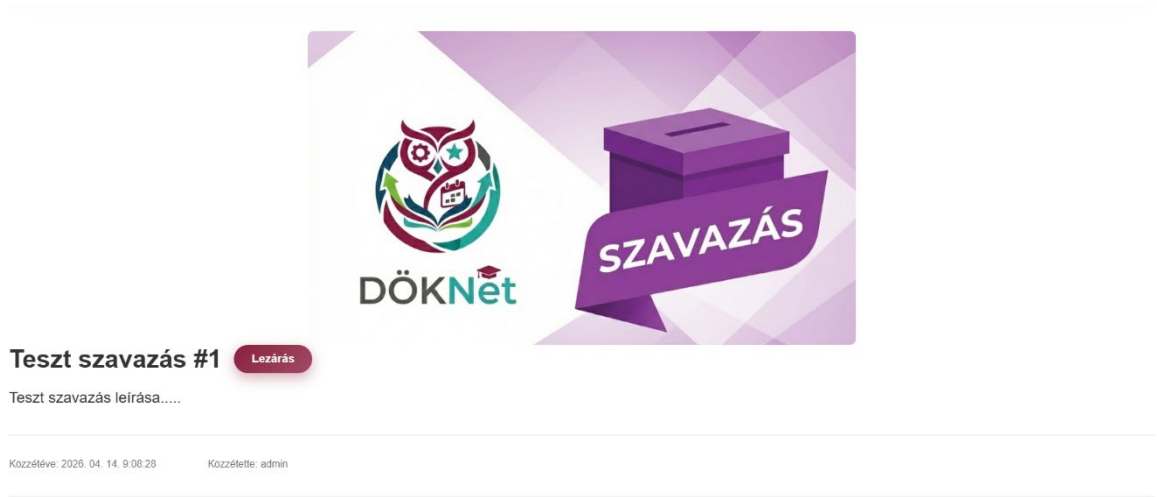
A szavazások létrehozása fülön címet és leírást lehet adni a szavazásnak. Ezek sima `input` és `textarea` mezőkkel lett létrehozva. Lehetőség van dinamikusan opció lehetőségeket létrehozni. Minimum 2 opciónak lennie kell, az X gomb megnyomásával lehet törölni a feleslegesnek gondolt opciókat. A mentés gombbal töltődik fel adatbázisba a szavazással kapcsolatos mindent adat. A szavazásoknál nincs lehetőség a borítókép kiválasztására, itt automatikusan megy a borítókép hozzáadás.



22. ábra: Szavazatok létrehozása

3.5.3. Elkészült szavazás

Az elkészült szavazások dizájnja az elkészült hírek stílus jegyeit hasznosította újra, azzal a pár apró változtatással, hogy nincs szerkesztési, illetve törlési lehetőség, helyette egy lezárás gombot láthat a megfelelő jogosultsággal rendelkező felhasználó (`Admin`, `Tanár`, `Elnök`). További változás a hírek panelhez képest, hogy a leírást követően nem reakció és kommentelési opciók vannak, hanem a választható opciók, amik mutatják a felhasználónak, hogy az adott pillanatban mennyi szavazatot kaptak a választható lehetőségek.



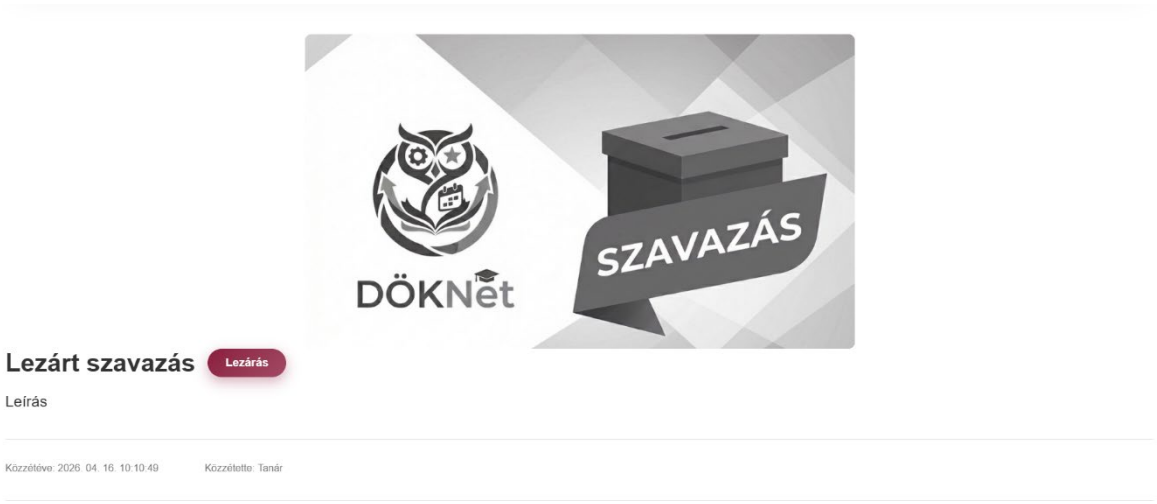
23. ábra: Kész szavazás eleje

24. ábra: Kész szavazás opciói

3.5.4. Lezárt szavazás

Ha egy szavazás le van zárva az azt jelenti, hogy az adatbázisban a votes táblában az `is_active` értéke 0-ról 1-re változott. Azért, hogy a lezárt szavazás minél látványosabban elkülöníthető legyen a még aktívaktól, a borítóképet megváltoztattuk egy szürke verzióra.

A lezárt szavazásoknál már nem lehet új szavazatot leadni mivel az opciók gombja a lezárást követően `disabled` lesz és továbbá az opciók felé kiírja a végeredményt is, azt, hogy melyik opció nyert és hogy arra mennyien szavaztak. Döntetlen esetén pedig azt írja ki, hogy melyik a kettő vagy több opció amire ugyanannyi szavazat érkezett

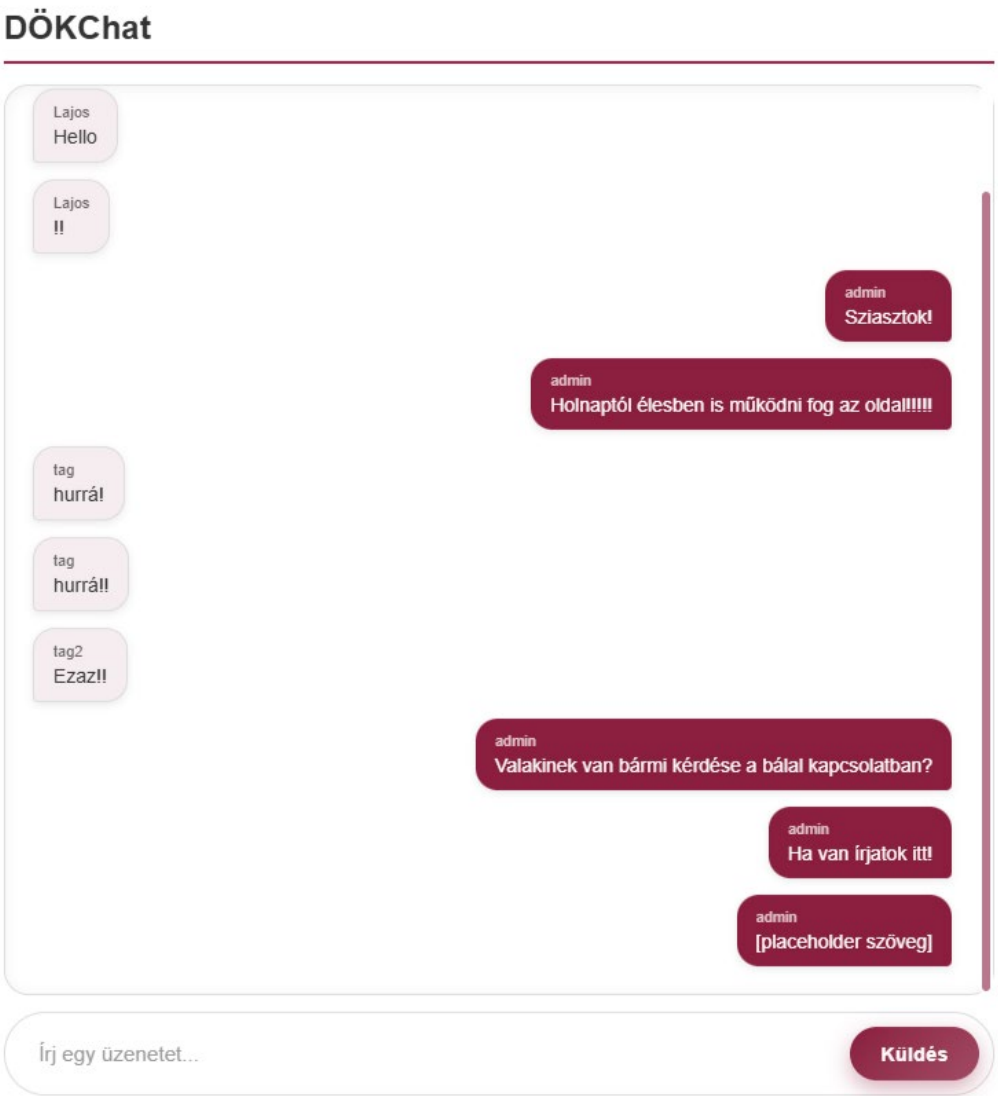


25. ábra: Lezárt szavazás

26. ábra: Lezárt szavazás

3.6. DÖKChat

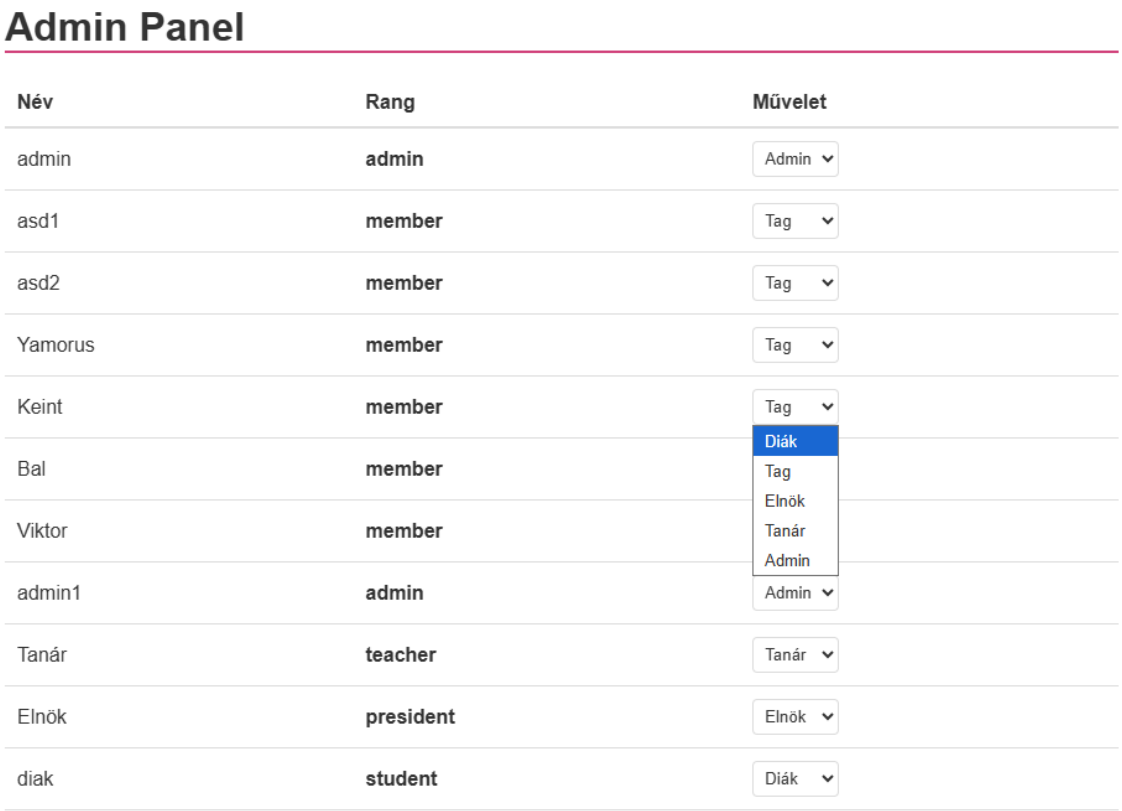
A chathez a `diák` felhasználókon kívül (`Admin`, `Tag`, `Elnök`, `Tanár`) férhetnek csak hozzá. Ezek a felhasználók tudnak üzeneteket küldeni a chatben. A chat célja az, hogy megkönnyítsük vele a kommunikációt a diákönkormányzat tagjai számára. A chat üzeneteit az adatbázisban tároljuk. A chaten nem tudunk üzeneteken kívül mást küldeni (pl. kép, fájl, hangüzenet). A chat nem tud értesítést küldeni a felhasználónak, csak akkor láthatjuk az üzeneteket, ha megnyitjuk a felületen. A chaten láthatjuk az üzenetküldő nevét, alatta magát az üzenetet. A chat automatikusan frissül három másodpercenként.



27. ábra: Chat

3.7. Admin panel

Az Admin panel célja az, hogy az `Admin` könnyebben tudja állítani a rangokat, mert ha az oldalra új felhasználó regisztrál az automatikusan `diák`-ként lesz elmentve. Az Admin panel kizárólag csak az `Admin` jogosultsággal rendelkező tagoknak jelenik meg az oldalon, a többi felhasználónak nem jelenik meg a `navbar`-ban ez a funkció. Magában a panelbán látható a felhasználó neve, jelenlegi rangja, és egy művelet funkció. Itt egy legördülő menü segítségével tudja az `Admin` átállítani az adott felhasználó rangját.



28. ábra: Admin Panel

4. Regisztráció/Bejelentkezés

A felhasználó kezelést `localStorage`-vel valósítottuk meg, ez azt jelenti, hogy a felhasználókat globálisan kezeljük az oldalon. `GlobalContext.tsx` biztosítja, hogy minden komponens elérje ezt a fájlt. Ebben a fájlban van lekezelve a sötét és világos mód is és annak váltási lehetősége. Ennek a fájlnak köszönhetően, ha a felhasználó az adott ezközön már belépett akkor nem szükséges az oldal újbóli felkeresése után ismételten bejelentkezni, a téma választás hasonló elven működik.

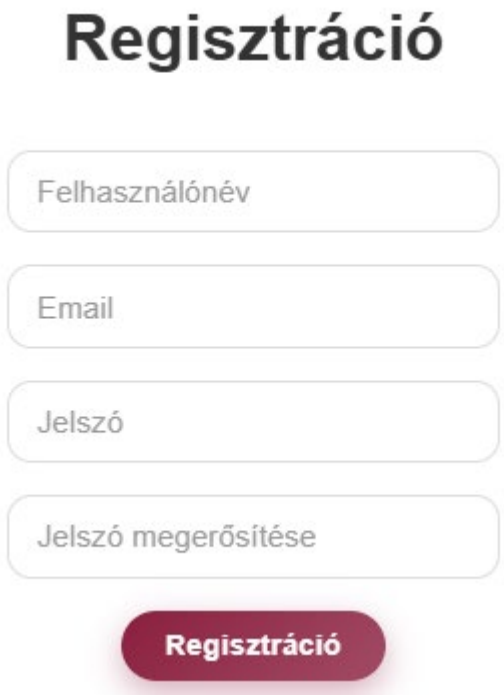
```
export const GlobalContext = createContext<GlobalContextType | null>(null);

export function GlobalProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<User>(null);
  const [theme, setTheme] = useState<string>("light");

  useEffect(() => {
    const storedUser = localStorage.getItem("user");
    setUser(storedUser ? JSON.parse(storedUser) : null);

    const savedTheme = localStorage.getItem("theme") || "light";
    setTheme(savedTheme);
    document.documentElement.setAttribute("data-theme", savedTheme);
  }, []);
```

29. ábra: GlobalContext

4.1. Regisztráció

A regisztrációs fülön lehet a felhasználónak saját fiókot készítenie. A felhasználó alapvetően vendégként van jelen, amivel nem tud hozzáférni több funkcióhoz sem. A regisztrációt követően a felhasználó először csak diák jogosultságot kap, ezt az `Admin` tudja később átállítani a megfelelőre (pl: `Tag`).

A regisztrációhoz kell adni egy felhasználó nevet egy email címet, ami jelenleg nincs használva, de tároljuk ezt is adatbázisban, szükséges továbbá megadni egy jelszót és annak ismételt megadását. Az oldal ellenőrzi, hogy a jelszó helyesen lett-e beírva mind a két alkalommal. A jelszót `bcryptjs` nevű kriptográfiai könyvtár egy egyirányú `hash`-sé alakítja és az kerül eltárolásra az adatbázisba.

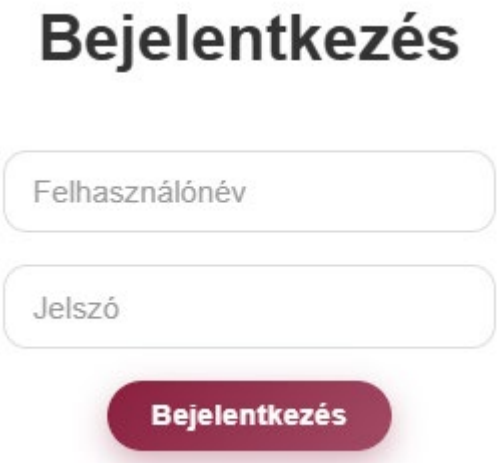


30. ábra: Regisztráció

4.2. Bejelentkezés

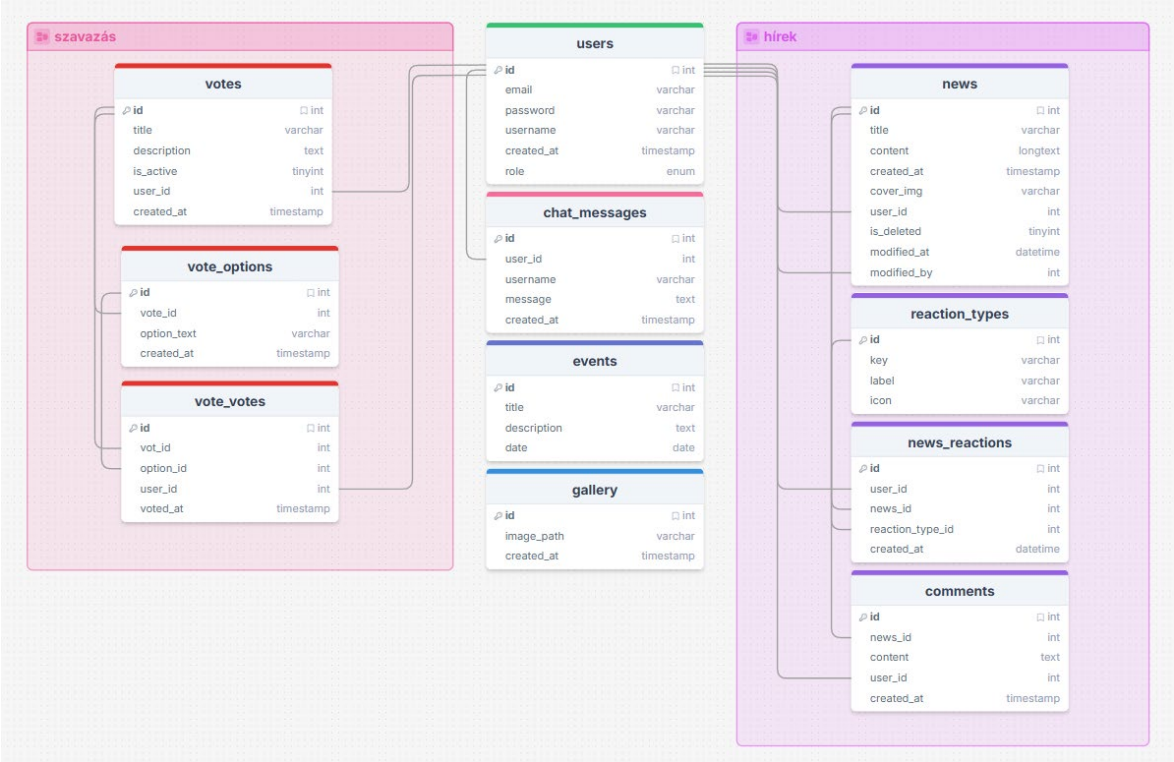
Itt lehet bejelentkezni már létező fiókba. Bejelentkezéshez csak egy felhasználónevet és egy jelszót kell megadni. A „Bejelentkezés” gomb megnyomását követően a backend ellenőrzi, hogy a megadott felhasználónév létezik-e és hogy a beírt jelszó megegyezik-e az adatbázisban elmentettel.

Ha ezek közül bármelyik érték hibás vagy nem létezik a `react-toastify` üzenet formájában értesíti a felhasználót a hibáról. Bejelentkezéskor a `bcryptjs` ellenőrzi, hogy a beírt jelszó `hash`-e megegyezik-e az adatbázisban már korábban elmentésre került `hash`-sel.



31. ábra: Bejelentkezés

5. A DÖKNet adatbázisa



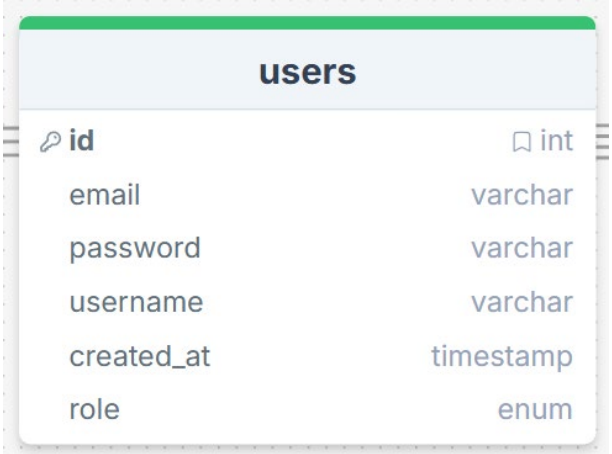
32. ábra: A teljes adatbázis

Az adatbázisunk 11 táblából áll, amik 2 nagyobb csoportból állnak, `szavazás` és `hírek` és 3 egyéb tábla, ami egyik csoportba se sorolható, illetve a `user` tábla, ami majdnem minden táblával kapcsolatban van. A `user` tábla az adatbázisunk legfontosabb táblája mivel ez kapcsolja össze a felhasználókat az egyes funkciókkal.

A táblákat úgy terveztük, hogy modulárisak legyenek, azaz, hogy a jövőben könnyen bővíthető legyen és ne legyen szükség szerkezet szintű módosításra.

Az adatbázis minden tábláját a phpMyAdmin oldalán keresztül készítettük. A grafikus felhasználói felület sokban megkönnyítette az egyes táblák létrehozását és kidolgozását.

5.1. users

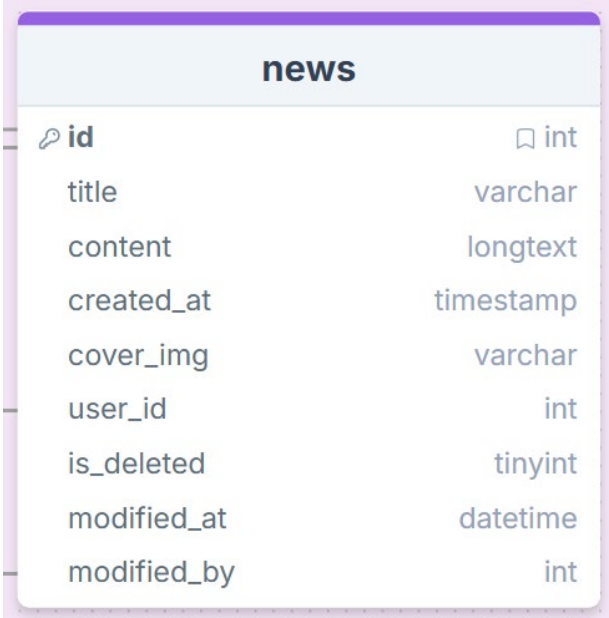


33. ábra: users tábla

Az adatbázisunk legfontosabb táblája a `users` tábla, itt kerül el tárolásra minden felhasználóval kapcsolatos adat. Az `id` a felhasználó saját egyedi azonosítója, a `username` az, ami a menü sávon és a chaten is megjelenik, illetve a hír és szavazások alatt, mint az adott médium közzétevője. A `created_at` tárolja a pontos idejét annak, hogy mikor lett létre hozva a felhasználó fiókja. A `role` tárolja a felhasználó jogosultságát ez lehet `student` - Diák, `member` - Tag, `president` - Elnök, `teacher` - Tanár, `admin` - Admin. A felhasználó a fiók létrehozását követően először diák szerepkört kap ezt azonban később az Admin az Admin panelen módosíthatja így hozzáférést kaphat több funkcióhoz is.

A `users` tábla `id` nevű oszlopa összeköttetésben áll a `votes` tábla `user_id` nevű oszlopával, a `vote_votes` tábla `user_id`-jával, a `chat_messages` tábla `user_id`-jával, a `news` tábla `user_id`-jával és a `modified_by` oszlopával, a `news_reaction` tábla `user_id`-jával, a `comments` tábla `user_id`-jával.

5.2. news



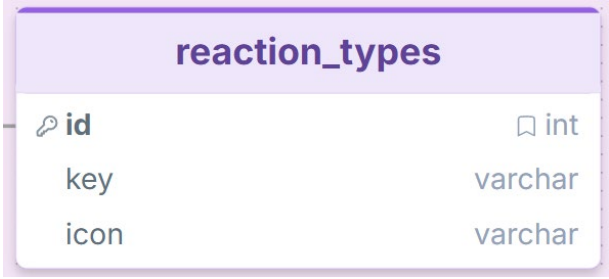
34. ábra: news tábla

A `news` tábla a `hírek` csoport tagja és egyben az adatbázis legösszetettebb táblája. Ebben tároljuk el a megírt hírek adatait.

Az `id` a megírt hír azonosító száma. A `title` a hír címe, ez jelenik meg nagyban a főoldalon is. A `content` a hír tartalmát tárolja. A `created_at` a hír létrehozásának idejét tárolja. A `cover_img` a 3 előre definiált borítókép elérési útvonalát tárolja. A `user_id` a hírt létrehozó felhasználó `id`-ját tárolja. Az oldalon ténylegesen nem lehet törölni a híreket csak a nem kívánt híreket nem jelenítjük meg ebben segít az `is_deleted` sor, aminek az értéke, ha 0 akkor az a hír megjelenik, ha 1 akkor az a hír nem érhető el semmilyen módon. A híreket van lehetőség szerkeszteni, a szerkesztés után eltároljuk a szerkesztés idejét a `modified_at`-ban, illetve a szerkesztést végző felhasználó id-ját a `modified_by`-ban.

A `news` tábla kapcsolatban áll a `users` táblával. Az `id` oszlopa a `news_reactions` tábla `news_id`-jával és a `comments` tábla `news_id`-jával.

5.3. reaction_types



35. ábra: reaction_types tábla

A `reaction_types` tábla a `hírek` csoport tagja. Feladata, hogy egységesen kezelje a hírekre adható reakciók fajtáit.

Az `id` a választható opciók egyedi azonosítóját jelöli, A `key` oszlop egy rövid egy szavas magyarázata a választható opcióknak pl: love, like, sad. Az `icon` oszlop elemei mind emojik, ezek jelennek meg a megnyitott híreknél is és ezek az adható reakciók.

A táblában 5 darab előre definiált értéket tárulunk, ezek az opciók azonban később még bővítésre kerülhetnek mivel az adatbázis struktúra lehetővé teszi ezt. Ez rugalmasabbá teszi a rendszert mivel a bővítéskor nem kell a tábla struktúrájában szerkezeti módosítást végrehajtanunk.

A `reaction_types` tábla `id` mezője kapcsolatban áll a `news_reactions` tábla `reaction_types_id` nevű mezőjével.

5.4. news_reactions

36. ábra: news_reactions tábla

A `news_reactions` tábla a `hírek` csoport tagja. Ez a tábla felelős az adott hírekre adott reakciók tárolásáért.

Az `id` a felhasználó által adott reakció egyedi azonosítója. A `user_id` annak a felhasználónak az azonosító száma, aki a reakciót adja a hírre. A `news_id` annak a hírnek az `id`-ja amire az adott reakciót adta a felhasználó. A `reaction_type_id` az 5 reakció közül kiválasztott emoji `id`-ja.

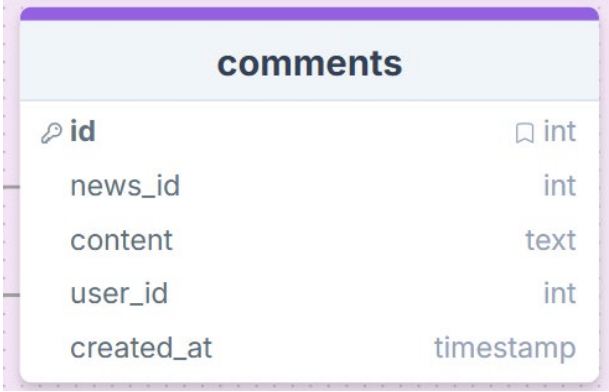
A tábla célja, hogy a hírekre adott reakciók adatait elkülönítve tárolja a hírektől, mivel egy hírre több felhasználó is adhat reakciót. Ezzel a felépítéssel átláthatóbbá és könnyebben értelmezhetővé teszi az adatbázisunkat.

A `news_reactions` tábla `user_id` nevű mezője kapcsolatban van a `users` tábla `id` mezőjével. A `news_id` kapcsolatban áll a `news` tábla `id` mezőjével. A `reaction_type_id` kapcsolatban van a `reaction_types` tábla `id` mezőjével.

Az adatbázist úgy építettük fel, hogy adatkonzisztencia legyen, azaz az adatok helyesek és egymással összhangban vannak. Nem létező felhasználó nem tud reagálni hírre, illetve nem létező hírre sem lehet reagálni.

Nagy hangsúlyt fektettünk arra, hogy elkerüljük az adat duplikációt igy megkönnyítve az adatok kezelését, lekérdezését, továbbá a, elősegítette az adatbázis optimalizáltságát.

5.5. comments



37. ábra: comments tábla

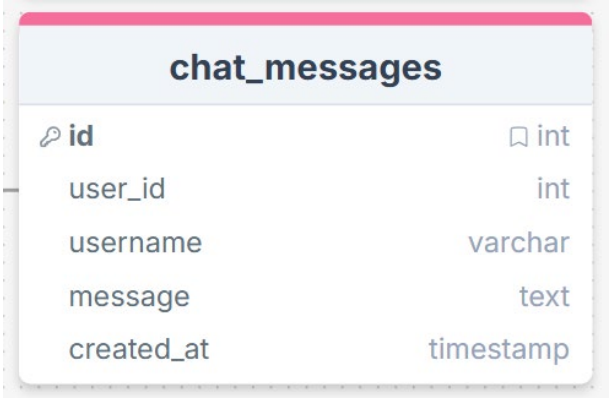
A `comments` tábla a `hírek` csoport tagja. Ennek feladata az egyes hírekhez írt hozzászólások adatainak kezelése és tárolása.

Az `id` mező a hozzászólások egyedi azonosítója. A `news_id` azt a hírt jelöli amire a hozzászólás érkezett. A `content` mező tárolja a hozzászólás tartalmát. A `user_id` a hozzászólást író felhasználó egyedi azonosítója. A `created_at` a hozzászólás közzételének a pontos idejét rögzíti.

A tábla célja, hogy az adott hírre érkező hozzászólásokat elkülönítse a `news` táblától így átláthatóbbá téve azt, mivel egy hírre több hozzászólás is érkezhet akár több felhasználótól is de akár ugyanattól az egytől is.

A `comments` tábla `news_id` mezője kapcsolatban áll a `news` tábla `id` mezőjével. A `user_id` össze van kapcsolva a `users` tábla `id` mezőjével.

5.6. chat_messages



38. ábra: chat_messages tábla

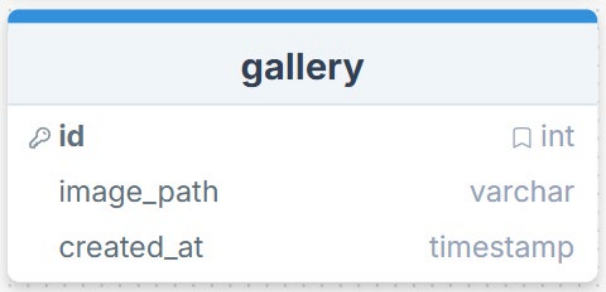
A `chat_messages` tábla nem tartozik a fő csoport egyikébe sem ez egy önálló funkcionális tábla. A tábla fő feladata, hogy eltárolja a `chatba` küldött üzeneteket.

Az `id` mező az üzenet `id`-ját tárolja. A `user_id` az üzenetet küldő felhasználó egyedi azonosítóját rögzíti. A `username` mező a `users` táblában lévő felhasználónevet használja. A `message` mező magát az üzenetet tartalmát tárolja el. A `created_at` mező az üzenet létrejöttének pontos dátumát, idejét tárolja.

A username mező tárolása lehetővé teszi az adatok gyorsabb megjelenítését, azonban ez bizonyos mértékű adatduplikációt eredményezhet.

A `chat_messages` tábla összeköttettetésben van a `users` tábla `id` mezőjével.

5.7. gallery



39. ábra: gallery tábla

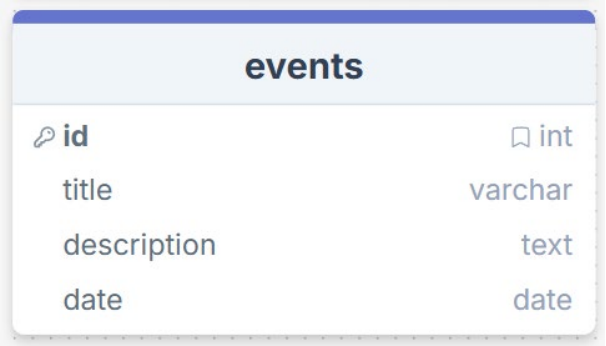
A `gallery` tábla nem tartozik a fő csoport egyikébe sem egy média-kezelő tábla, a rendszerben feltöltött képek elérési útját tárolja. A tábla nem bináris képadatokat (BLOB) tárol, hanem kizárólag a fájlokra mutató hivatkozásokat, így a tényleges képfájlok az uploads mappában kerülnek elhelyezésre.

Az `id` mező a kép egyedi azonosítóját jelöli. Az `image_path` a feltöltött kép elérési útvonalát tárolja. Ez nem magát a képfájlt tartalmazza. A `created_at` mező a kép feltöltésének pontos időpontját rögzíti timestamp-el, ez lehetővé teszi a képek időrend szerinti rendezését, valamint a feltöltési aktivitás követését.

A tábla szándékosan minimalista, mivel nincs szükség metaadatokra, nem tárolunk semmi féle adatot arról a felhasználóról, aki a képeket feltölti, a tábla egyetlen feladata, hogy a kép megjelenítéséhez szükséges elérési utat tárolja.

Azért nem bináris adatként tároltuk a képeket mert így kisebb az adatbázis mérete, nem lassítja a lekérdezéseket. Egyszerűbb ezzel a módszerrel az adatbázis backupja bármiféle hiba esetén, valamint könnyebb a hordozhatósága.

5.8. events



40. ábra: events tábla

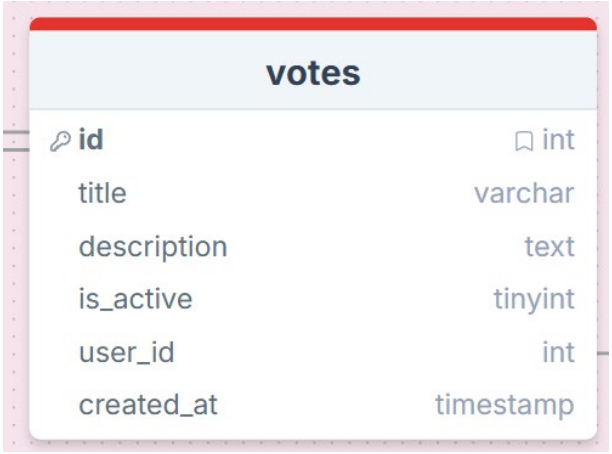
Az `events` tábla nem tartozik a fő csoport egyikébe sem ez egy önálló funkcionális tábla. Az oldalon lévő naptár eseményeinek eltárolására szolgál.

Az `id` mező az esemény azonosítóját tárolja. A `title` mező az esemény nevét/címét rögzíti. A `description` mező az adott esemény részletes leírásának tárolására szolgál. A `date` mező pedig az esemény időpontját (csak dátumot) tárolja el.

A tábla feladata az, hogy a lehető legegyszerűbb és legkönnyebb módon tárolja a naptár eseményeit.

Ez az adatbázis 2 olyan táblájának egyike, amelynek nincs semmilyen kapcsolata a többi táblával. Ez azonban később még változhat, például szeretnénk majd rögzíteni az eseményt közzétevő felhasználó azonosítóját.

5.9. votes



41. ábra: votes tábla

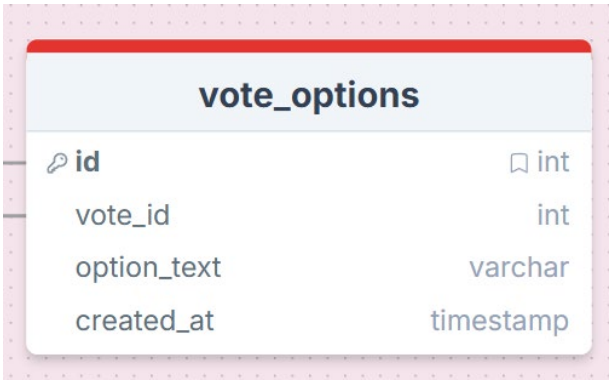
A `votes` tábla a második nagy csoport a szavazások tagja. Ennek a táblának az a fő feladata, hogy kezelje és tárolja a készített szavazások adatait.

Az `id` mező az egyes szavazások egyedi azonosítója. A `title` mező a szavazás címét tárolja el. A `descripiton` a szavazás leírását, részletes magyarázatát tárolja. Az `is_active` az `is_delete`-hez hasonlóan vagy 0, vagy 1 értékkel rendelkezik, ha 0 akkor a szavazás még aktívnak számít, ha 1 akkor le van zárva és nincs többé lehetőség arra szavazatot leadni. A `user_id` a szavazást létrehozó felhasználó azonosítóját tárolja. A `created_at` a szavazás közzétételének időpontját tárolja el.

A `votes` tábla `id` mezője kapcsolatban áll a `vote_options` tábla `vote_id` mezőjével és a `vote_votes` tábla `vote_id` mezőjével, illetve a `user_id` összeköttetésben áll a `users` tábla `id` mezőjével.

A tábla központi szerepet tölt be a szavazási rendszerben, mivel ehhez kapcsolódnak az opciók és a leadott szavazatok.

5.10. vote_options



42. ábra: vote_options tábla

A `vote_options` tábla a szavazás csoport tagja. Feladata a szavazás létrehozásakor megírt szavazati opciók adatainak kezelése és tárolása az adatbázisban.

Az `id` mező az egyes opciók egyedi azonosítója. A `vote_id` annak a szavazásnak az azonosítóját határozza meg amihez az opciót írták, ez az érték duplikálódhat mivel egy szavazásra több opciót is ír a felhasználó. Az `option_text` az adott opciók tartalmát tárolja. A `created_at` az opciók létrehozásának idejét tárolja el.

A `vote_options` tábla `id` mezője kapcsolatban van a `vote_votes` tábla `option_id` nevű mezőjével, továbbá a `vote_id` össze van kapcsolva a `votes` tábla `id` mezőjével.

Az opciók külön táblában történő tárolása átláthatóbbá teszi az adatbázist, és megkönnyíti a szavazások kezelését.

5.11. vote_votes

43. ábra: vote_votes tábla

A `vote_votes` tábla a szavazás csoport tagja. Feladata a létező és aktív szavazások esetében rögzíteni a felhasználót és azt, hogy melyik szavazásnál melyik opciót választotta.

Az `id` a felhasználó által leadott szavazatok egyedi azonosítója. A `vote_id` annak a szavazásnak az azonosítója amire a felhasználó szavazata érkezett. Az `option_id` a kiválasztott opció azonosítója. A user_id annak a felhasználónak az azonosítója, aki szavazatot adott le. A voted_at rögzíti azt az időpontot amikor a szavazat le lett adva.

A `vote_votes` tábla `vote_id` mezője kapcsolatban áll a `votes` tábla `id` mezőjével, továbbá az `option_id` az `vote_options` tábla `id` mezőjével van összekapcsolva

A szavazás csoport tábláit direkt úgy építettük fel, hogy jól strukturált legyen és az egyes feladatoknak megfelelően külön legyenek választva ezzel is az átláthatóságát elősegítve.

6. A weblap designja

A weblap arculata letisztult, egyszerű és átlátható. Felhasználtuk az iskolánk színeit a weboldalhoz, hogy egységes kinézete legyen minden oldalnak és tükrözze magát az iskolát és amit képvisel. Vizuálisan egységes az oldal. Az oldalak betöltésekor animáció megy végbe, elérhető opció a felhasználók számára a sötét módra váltás is, amely egy hasznos kényelmi funkció. Minden oldal aljára elhelyeztünk egy láblécet, amely az oldallal és az iskolával kapcsolatos információkat tartalmaz. A menünek és a gomboknak lekerekített forma világa van. Az oldal betűstílusa `Arial`. A weboldal arculatának kidolgozásakor fontos volt számunkra az egyszerűség, de ugyanakkor a praktikusság.

6.1. Logó

Az oldal elsődleges azonosítója a DÖKNet logó, amely egy bölcsességet szimbolizáló, színes bagoly motívumot ábrázol, a bagoly régóta a bölcsesség, tudás és tanulás szimbóluma. Mivel a DÖKNet egy diákönkormányzati oldal, ez a szimbólum hangsúlyozza az oktatási környezethez való tartozást és a tudás megszerzésének fontosságát.

A logó színei és dinamikus vonalvezetése a diákönkormányzat fiatalos energiáját tükrözi. A baglyot körülölelő nyilak, vagy egymásba fonódó ívek a folyamatos fejlődést, az események áramlását és a közösség dinamizmusát jelképezik. Ez utalhat a rendezvények szervezésére és az azokon való részvételre is. A körforma a teljességet, az egységet és a közösséget is erősíti.

A bagoly egyik szeménél látható fogaskerék az eseményszervezés, a működés, a logisztika és a hatékonyság jelképe.

A bagoly másik szeménél lévő csillag a siker, a kitűnőség, az inspiráció és a diákok számára nyújtott lehetőségek szimbóluma.

A bagoly testébe integrált naptárikon egyértelműen a rendezvényszervezésre, időpontok kezelésére és a diákok értesítésére utal.

A logónk az oldal több helyén is elhelyezésre került többek között a menüsáv bal felső sarkában és a főoldalon.

44 ábra: DÖKNet logó

6.2. Szín paletta

Az oldal háttérszíne a fehér ennek hex kódja a `#ffffff`, ami az egyszerűséget és modernséget tükrözi, ezért választottuk ezt az oldalunk legvisszatérőbb és legtöbbet használt színének.

45 ábra: Háttérszín

A gombok színei #8b1e3f, ami egy bordós szín ezt a logónk és a z iskola bordós dizájnja miatt választottuk az oldalra, ezzel is azt szerettük volna, ha még jobban az iskolához köthető lenne az oldalunk.

46 ábra: Gombok színe

Bizonyos oldalrészeken megjelenik még egy fajta gomb szín ez a #2575fc, ez a szín 3 helyen jelenik meg az oldalunkon. A naptár szerkesztésénél a „mégse” gomb

színe, a szavazás lezárásánál szintén, illetve a hírek szerkesztésénél is ezt a feladatot ellátó gomb színe.

47 ábra: „mégse” gombok színe

Az oldalon szinte minden gomb lekerekített stílusú, egyedül a navbar-ban találhatóak más gombok, ezek hover-jai is bordó színűek. Az oldalon próbáltunk arra törekedni, hogy a színek nagyjából tükrözzék a logót, de ne vigyen bele túl sokat, ezért is döntöttünk a bordó szín mellett. Az oldal inputjainál is jelen van a bordó szín, itt a keret lesz bordó, valamint halványan olyan színűvé is válik.

6.3. Gombok, inputok

A gombok és input mezők kialakításánál fontos szempont volt az egységesség és a könnyű felismerhetőség. A lekerekített formák és az egyszerű, jól elkülöníthető színek segítik a felhasználót abban, hogy gyorsan azonosítsa az egyes funkciókat. A gombokon elhelyezett szövegek egyértelműen jelzik azok működését, így csökkentve a félreértés lehetőségét.

Az interaktív elemeknél külön figyelmet fordítottunk a vizuális visszajelzésekre is. A gombok hover állapotban színváltozással vagy enyhe kiemeléssel reagálnak, ami segíti a felhasználót a navigáció során. Az input mezők fókusz esetén szintén kiemelést kapnak, így egyértelműen látható, hogy éppen melyik mező aktív.

Az input mezők kialakítása során az egyszerűség és az átláthatóság volt a fő cél. A megfelelő méret és a jól látható keret biztosítja, hogy a felhasználó kényelmesen tudjon adatokat bevinni. A form elemek egységes megjelenése hozzájárul ahhoz, hogy az oldal professzionális és rendezett benyomást keltsen.

6.4. Reszponzív kialakítás

A weboldal reszponzív kialakítással készült, ami azt jelenti, hogy a felület automatikusan alkalmazkodik a különböző képernyőméretekhez. Ennek köszönhetően az oldal nemcsak asztali számítógépen, hanem mobiltelefonon is jól

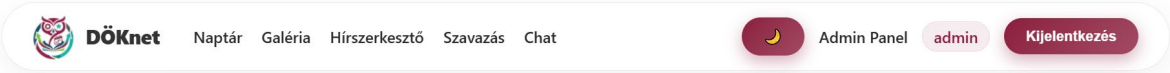
használható marad. Az elemek elrendezése rugalmasan változik, így a tartalom minden eszközön átlátható és könnyen kezelhető.

6.5. Navigáció és felhasználói élmény

A weboldal navigációját úgy alakítottuk ki, hogy a felhasználók minél gyorsabban és egyszerűbben elérjék a számukra fontos funkciókat. A felső menüsáv jól átlátható struktúrát követ, a menüpontok logikusan vannak csoportosítva.

A menüsávot úgy alakítottuk ki, hogy az egyes funkciókat csak az arra feljogosított felhasználók láthatják, például az `Admin` minden funkcióhoz hozzáfér, míg a `Diák` csak a főoldal, a naptárat és a galériát tudja használni, de ezeket is csak korlátozottan

A felhasználói élmény javításának érdekében a `react-toastify`-t alkalmazzuk, hogy azonnali visszajelzést kapjanak a felhasználók.



48 ábra: A DÖKNet menüje

7. Frontend

7.1. Technológiai alapok

A frontend fejlesztéséhez a React alapú Next.js keretrendszert használtuk, mert ennek használatával képesek voltunk gyors és modern felhasználói felületeket létrehozni.

A Next.js-t főként azért választottuk mert egy full-stack keretrendszer és a routing rendszere egyszerű és könnyen tanulható.

A React komponens alapú felépítése segítette a fejlesztést azzal, hogy a hasonló UI elemeket különálló újrafelhasználható komponensekbe szerveztük ki, ezzel gyorsítva a saját munkánkat. Ez továbbá átláthatóbbá teszi a kódot és lehetőséget ad a későbbi bővítésre is.

TypeScriptet használtunk a fejlesztéskor a típusellenőrző sajátossága miatt. Ez növeli a kód olvashatóságát és megbízhatóságát.

A React lehetőséget biztosít a kliens oldali működés során az interaktív felhasználói élmény kialakítására, ahol az adatok változására a felület automatikusan frissül, ezért nincs szükség az oldal újratöltésére.

7.2. Komponens alapú felépítés

A projektmunka során több saját komponenst is használtunk a felhasználói felület készítéséhez. Például a `NavBar` esetében, ami a navigációs sávot jeleníti meg az oldal minden részén, a főoldalon a híreket kártyákban megjelenítő komponens a `NewsList`, a híreken belüli reakciók adására lehetőséget biztosító komponensünk a `Reactions`.

A hírek törlésére, a naptárban módosítani kívánt esemény szerkesztője és a szavazások lezárására modális ablak komponenseket használtunk.

A komponensek külön fájlokban kerültek megvalósításra, ez megkönnyítette a fejlesztés menetét és a kódok átláthatóságát. A rendszerben az adatkezelés főként API hívásokon keresztül történik, ahol az egyes komponensek saját maguk töltik be a szükséges adatokat a backendről.

A React komponensek közötti adatátadás alapvetően props segítségével valósulhat meg, amely lehetővé teszi, hogy a szülő komponensek adatokat és függvényeket adjanak át a gyermek komponenseknek. A jelenlegi implementációban azonban a hangsúly inkább az önálló adatlekérést végző komponenseken van, nem pedig a központi state-ből props-on keresztüli adatmegosztáson.

A komponensalapú architektúra egyik legnagyobb előnye a modularitás, amely lehetővé teszi, hogy a felhasználói felület kisebb, jól elkülönített egységekre bontva kerüljön megvalósításra. Az újrafelhasználható komponensek használata csökkenti a kódismétlést, valamint gyorsítja a fejlesztési folyamatot, mivel ugyanaz a kód logika több helyen is alkalmazható. Az architektúra további előnye az átláthatóság, mivel a külön fájlokba szervezett komponensek jól strukturált projektstruktúrát eredményeznek.

A komponenseket úgy írtuk meg, hogy minden komponens egy elkülöníthető feladatot lát el ezzel is a projekt.

7.3. Állapotkezelés

A frontend állapotkezelése React hookok segítségével valósul meg, amely lehetővé teszi a komponensek belső adatainak és a felhasználói interakciók kezelését.

A projekt készítése során legfőképpen 2 React hookot alkalmaztunk a useStatet és a useEffectet.

A useState a komponensek lokális állapotainak kezelésére szolgál, például adatok tárolására és a felhasználói felület dinamikus frissítésére.

A useEffect az API hívások futtatásáért felel, ez a komponens betöltődésekor automatikusan lefut.

Az oldalon továbbá még használjuk a useContext hookot is a globális adatok kezelésére, ez teszi lehetővé a felhasználói és alkalmazás szintű beállítások megosztását fájlok között.

A projekt készítése során nem került alkalmazásba 4 fajta React hook. A useReducer komlex állapot logikák kezelésére alkalmas. A useMemo és a useCallback a teljesítmény optimalizálásra használható hookok, gyakran újra renderelt komponensek vagy fájlok esetében. A useRef közvetlen DOM manipulációra vagy perzisztens értékek tárolására használható.

A jelenlegi implementációban ezekre a hookokra nem volt szükség, mivel az alkalmazás állapotkezelési igényei viszonylag egyszerűek és jól kezelhetők a useState és useEffect kombinációjával.

8. Backend

8.1. Backend architektúra és működési modell

A DÖKNet rendszer backendje a kliens–szerver architektúra elvén működik. Ez azt jelenti, hogy a felhasználói felület és a szerveroldali logika elkülönül egymástól, és a kommunikáció közöttük hálózati kéréseken keresztül történik. A kliens felelős a felhasználói interakciók kezeléséért és az adatok megjelenítéséért, míg a szerver az üzleti logikát, az adatfeldolgozást és az adatbázis-kezelést végzi.

A projektet Next.js-ben készítettük, ami egy full-stack keretrendszer, ez teszi lehetővé, hogy a frontend és a backend is egyetlen alkalmazásban legyen megvalósítva. A szerver oldali logika a Next.js projekt részeként fut.

A backend funkciókat úgynevezett API route-ok segítségével valósítjuk meg. Ezek olyan szerveroldali végpontok, amelyek a projektben külön fájlokként jelennek meg, és HTTP kéréseket képesek fogadni. Minden API route egy adott funkcióhoz kapcsolódik, például hírek lekéréséhez, szavazások kezeléséhez vagy reakciók feldolgozásához. A frontend ezekre a végpontokra küld kéréseket, a backend pedig feldolgozza azokat és választ küld vissza a frontendnek.

A rendszer működése a request–response ciklusra épül. Ez azt jelenti, hogy a kliens elküld egy HTTP kérést a szerver felé, ami tartalmazza a szükséges adatokat. A szerver ezt a kérést feldolgozza, végrehajtja a szükséges műveleteket, majd visszaküld egy választ a kliensnek. Ez a ciklus minden interaktív műveletnél ismétlődik az alkalmazásban.

Az adatokat JSON formátumban küldjük el a kliens és a szerver között. A JSON egy könnyen olvasható és strukturált adatformátum, amely ideális webes alkalmazások számára. A frontend JSON formátumban küldi el az adatokat a backend felé, majd a backend szintén JSON válaszokat ad vissza, amiket a frontend feldolgoz és megjelenít a kliens oldalon.

Összességében a Next.js alapú kliens–szerver architektúra, az API route-ok használata, valamint a JSON alapú kommunikáció egy modern, jól strukturált és könnyen bővíthető rendszert eredményezett a DÖKNet számára.

8.2. API réteg és végpontok részletes működése

8.2.1. Bevezetés

A rendszer backendje több, funkció szerint elkülönített API végpontból áll, amelyek a DÖKNet különböző moduljait szolgálják ki. Minden végpont egy adott művelet kezeléséért felel, például felhasználói műveletekért, hírek kezeléséért, szavazási logikáért vagy interaktív funkciókért. A végpontok REST alapelvek szerint kerültek kialakításra, így erőforrás-alapú struktúrát követnek, és a HTTP metódusok (GET, POST, PUT, DELETE) segítségével valósítják meg a műveleteket.

8.2.2. A projekt API végpontjai csoportosítva

Felhasználó kezelés:

```
/api/admin
/api/login
/api/register
```

Hírek:

```
/api/news
/api/news-editor
/api/options
/api/reactions
/api/delete
```

Szavazás:

```
/api/vote
/api/vote_close
/api/vote-editor
```

Chat:

```
/api/chat
```

Galéria:

```
/api/gallery
/api/upload
```

8.2.3. Működési logika

A DÖKNet rendszer API végpontjai HTTP metódusokra épülnek, amelyek meghatározzák az adott kérés típusát és célját. A GET metódus adatlekérésre szolgál, a POST új adatok létrehozására, a PUT meglévő adatok módosítására, míg a DELETE adatok törlésére használatos. Ez a felosztás biztosítja a REST-szerű működés logikai struktúráját és áttekinthetőségét.

A kliens által küldött kérések a request body-ban kerülnek továbbításra JSON formátumban. Ez lehetővé teszi, hogy a frontend és a backend jól értelmezhető formában kommunikáljon egymással.

A beérkező adatok feldolgozása során a szerveroldali validáció kiemelten fontos szerepet kap. A backend minden esetben ellenőrzi a kapott adatokat, például azok típusát, meglétét és helyességét, mielőtt további műveleteket végezne velük. Ez csökkenti a hibák és inkonzisztens adatok bekerülésének lehetőségét az adatbázisba.

A rendszer hibakezelése HTTP státuszkódok segítségével történik, A 200-as kód a sikeres műveletet jelzi, a 400-as hibás vagy hiányos kérést, a 401-es jogosultsági problémát, míg az 500-as kód szerveroldali hibát jelez.

8.2.4. Kapcsolat a frontenddel

A frontend és a backend közötti kapcsolat API hívásokon keresztül valósul meg, amelyeket a frontend a fetch függvény segítségével hajt végre. A felhasználói műveletek során a frontend HTTP kéréseket küld a megfelelő backend végpontokra, például adatok lekérésére, létrehozására vagy módosítására.

A kommunikáció aszinkron módon történik, így a felhasználói felület nem akad meg a kérések feldolgozása közben sem. A backend a beérkező kérésekre JSON formátumban válaszol, amelyet a frontend dolgoz fel és jelenít meg a felhasználó számára.

Ez a felépítés lehetővé teszi a dinamikus adatkezelést és a gyors felhasználói élményt, mivel az oldalak újratöltése nélkül frissülnek az adatok. A frontend és backend közötti kapcsolat így jól strukturált, skálázható és könnyen karbantartható.

8.3. Adatkezelés és adatbázis műveletek

Az adatbázis-kezelés során a rendszer SQL műveleteket használ az adatok lekérdezésére és módosítására. A `SELECT` utasítás az adatok lekérdezésére szolgál, például hírek, felhasználók vagy szavazási eredmények megjelenítésekor. Az `INSERT` művelet új adatok rögzítését teszi lehetővé, például új felhasználó regisztrálásakor vagy új bejegyzések létrehozásakor, míg az `UPDATE` a meglévő adatok módosítására szolgál, például reakciók vagy szavazatok frissítése esetén. A rendszerben bizonyos esetekben a `PATCH` művelet is alkalmazásra kerül, amely részleges adatmódosítást tesz lehetővé. Ez azt jelenti, hogy nem a teljes rekord kerül felülírásra, hanem csak a megváltozott mezők frissülnek. Ez különösen hasznos olyan funkcióknál, ahol kisebb, célzott módosítások történnek.

```
const [result] = await db.query<ResultSetHeader>(
  "INSERT INTO news (title, content, cover_img, user_id) VALUES (?, ?, ?, ?)",
  [title, content, cover_img ?? null, user_id],
);
```

49. ábra: Hír INSERT

```
const [result] = await db.query<ResultSetHeader>(
  "UPDATE votes SET is_active=1 WHERE id = ? AND is_active = 0 ",
  [vote_id],
);
```

50. ábra: Szavazás lezárás UPDATE

Az adatstruktúra kialakításánál a normalizáció alapelveit is figyelembe vettük, amely csökkenti az adatismétlést és javítja az adatbázis szerkezetének hatékonyságát. A táblák logikailag elkülönített részekben tárolják az adatokat, ezáltal elkerülhető az inkonzisztencia.

Az adatkonzisztencia fenntartása érdekében a rendszer minden adatmódosításnál szerveroldali ellenőrzést alkalmaz, így biztosítva, hogy csak érvényes és megfelelő formátumú adatok kerüljenek az adatbázisba.

8.4. Hitelesítés és jogosultságkezelés

A DÖKNet rendszerben a hitelesítés és a jogosultságkezelés a backend logikájának egyik központi eleme, amely biztosítja, hogy az API végpontokhoz csak megfelelően azonosított és megfelelő jogosultsággal rendelkező felhasználók férhessenek hozzá.

A jelszavak kezelése során a rendszer a `bcryptjs` könyvtárat használja, amely egy kriptográfiai hash-elő algoritmus. A felhasználók jelszavai nem kerülnek nyers formában eltárolásra az adatbázisban, hanem a regisztráció során egy egyirányú hash értékké alakulnak. Ez azt jelenti, hogy az eredeti jelszó nem visszafejthető, így az adatbiztonság jelentősen növekszik még adatbázis-szivárgás esetén is.

Bejelentkezéskor a backend nem az eredeti jelszavakat hasonlítja össze, hanem a megadott jelszót újra hash-eli, majd ezt veti össze az adatbázisban tárolt hash értékkel. Ez képezi a login validáció backend logikáját, amely minden esetben szerveroldalon történik, így a hitelesítés nem kerülhet meg a kliens által.

A rendszer role-alapú jogosultságkezelést alkalmaz, ahol a felhasználókhoz különböző szerepkörök tartozhatnak. Ezek a szerepek határozzák meg, hogy az adott felhasználó mely API műveleteket érheti el. Minden fontos műveletnél a backend ellenőrzi a felhasználó szerepét, és csak akkor engedélyezi a végrehajtást, ha a jogosultság megfelelő.

A frontend oldalon a felhasználói állapotot a rendszer localStorage segítségével tárolja, azonban ez kizárólag kényelmi és felhasználói élményt javító célokat szolgál. A localStorage-ban tárolt adatok nem minősülnek biztonsági hitelesítésnek, mivel kliensoldalon módosíthatók. A valódi jogosultságellenőrzés minden esetben a backend feladata, így a rendszer biztonsága nem a kliensoldali tároláson múlik.

```
const hashedPassword = await bcrypt.hash(password, 10);
```

51. ábra: bcrypt

8.5. Hibakezelés, validáció és rendszerstabilitás

A DÖKNet rendszerének backendjét úgy terveztük, hogy nagy figyelmet fordítottunk a hibakezelésre és az adatok megfelelő validációjára, mivel ezek alapvető szerepet játszanak a rendszer stabil működésében. A hibák kezelése egységes logika mentén történik, amely biztosítja, hogy a rendszer ne álljon le váratlan szituációkban sem, valamint, hogy a felhasználók is megfelelő visszajelzést kapjanak.

A szerveroldali működés során a hibák kezelésére try-catch szerkezeteket használunk, amelyek lehetővé teszik a futás közben fellépő kivételek kezelését. Ilyen esetek lehetnek például adatbázis-műveletek hibái vagy nem várt bemeneti adatok. A hibák elkapása után a rendszer nem áll le, hanem kontrollált módon válaszol a kliensnek.

A beérkező adatok ellenőrzése minden fontosabb végpontnál megvalósul. A backend validálja az adatok meglétét, típusát és formátumát, mielőtt azokat feldolgozná vagy adatbázisba mentené. Ez különösen fontos olyan műveleteknél, mint a regisztráció vagy bejelentkezés, ahol a hibás bemenetek könnyen inkonzisztens állapothoz vezethetnek. A validáció segítségével a rendszer már a feldolgozás elején kiszűri a nem megfelelő adatokat.

A frontend a backend válaszaira építve jeleníti meg a visszajelzéseket, például toast értesítések formájában. Ez biztosítja, hogy a felhasználó azonnali és egyértelmű információt kapjon egy adott művelet eredményéről, legyen szó sikeres végrehajtásról vagy hibáról.

A rendszer tervezése során nagy hangsúlyt fordítottunk az úgynevezett edge case-ek kezelésére is. Ide tartoznak azok a ritkábban előforduló esetek, mint például hiányzó paraméterek, ismételt műveletek vagy nem létező adatokra történő hivatkozás. Ezek megfelelő kezelése hozzájárul a rendszer stabilitásához és kiszámítható működéséhez.

9. Tesztelés

9.1. Menete

A programot úgy teszteltük, hogy megkértük a diákönkormányzat elnökét és vezető tanárát, hogy teszteljék le a gyakorlatban, hogy egy DÖK-ös számára megfelelő élményt és kényelmet nyújt-e. Ők a tesztelés során folyamatosan elmondták a véleményüket, így rögtön láttuk, ha valahol hiba van, esetleg elakadtak vagy nem volt egyértelmű egy gomb funkciója.

9.2. Tapasztalatok

A tesztelők visszajelzései alapján a felület átlátható és jól kezelhető lett, azonban felmerült az igény egy sötét módra, és egy csak az admin által elérhető könnyen használható jogosultság kezelő panel, amiket később hozzá is adtunk az oldalhoz.

9.3. Konklúzió

A tesztelési fázis végére a program elérte azt a stabilitást és azt a szintű felhasználói élményt, amellyel készen áll az éles használatra a diákönkormányzati munka részére.

10. Későbbi fejlesztési lehetőségek

10.1. Chat

A chatet lehetne később bővíteni olyan funkciókkal, mint például hangüzenet küldése, fájlok csatolása stb. További lehetőség lehet, hogy az üzenet alatt, mellett vagy felett látni lehetne, hogy mikor küldték el az üzenetet (dátum, idő). Terveinkben szerepel még, hogy lehessen különböző csoportokat létrehozni a chaten az adott szervezendő eseménynek megfelelően vagy a dökön belüli azonos feladatokat ellátó személyek számára. Továbbá tervben van az is, hogy a chat értesítést küldjön a felhasználóknak egy újabb üzenet érkezéséről.

10.2. Naptár

A későbbiekben a naptáron keresztül lehetne jelentkezni olyan eseményekre, amik
jelentkezéshez vannak kötve, és ez visszajelzést küldene, hogy mennyi felhasználó jelentkezett az eseményre. Itt a diák felhasználók is tudnak visszajelzést küldeni, hogy mennyire érdekli őket. A dök tagok pedig tudnának hozzáfűzni javaslatokat az eseményekhez melyet az esemény létrehozója tudna elbírálni, hogy hozzáadja-e az eseményhez. Hasonló módon, mint a chatnél, itt is tervben van, hogy a naptár küldjön értesítést a felhasználóknak. Ez lehetne úgy is, hogyha közeledik egy esemény előtte küldene a program egy értesítést, de küldhet az adott napon is amikor az esemény van. Valamint a DÖK-ösök láthatnák a naptárban, hogy melyik eseményre hányan jelentkeztek, mi volt a legnépszerűbb, így könnyebb a jövő évi tervezés.

10.3. Galéria

A galériában tervezünk egyéb médiafájlok feltöltésének lehetőségét is mint például videók vagy akár gifek. Szeretnénk ezen kívül, ha lehetőség lenne kategorizálni külön az adott eseményekhez köthető mappákban az eseményekről készült képeket. Lehetne itt egy keresősáv is, hogy minél egyszerűbben meg tudjuk találni az eseményekről készült képeket, videókat. A galéria struktúráját is szeretnénk majd a későbbiekben lecserélni. Ahelyett, hogy az adatbázisban csak a

képek vagy később egyéb médiumok elérési útját tárolnánk, szeretnénk magát a fájlt eltárolni az adatbázisban.

10.4. Főoldal

A főoldalon láthatóvá válhatnának például a naptárban lévő közelgő események attól függően, hogy milyen közel van az adott esemény. A szavazással is megcsinálhatnánk ugyanezt. A képnézegetőben akár azt is meg lehetne valósítani, hogy mindig a legutóbbi esemény képei jelenjenek meg benne. Továbbá azt is meg lehetne csinálni, hogy a képek maguktól váltakozzanak automatikusan.

10.5. Szavazás

Lehetnének a diák felhasználók számára is szavazások, amikkel inkább azt lehetne felmérni, hogy mennyi érdeklődés van egy adott, még meg nem szervezett esemény iránt. Szeretnénk majd egy olyan oldal részt is, ahol azt is lehetne látni, hogy melyik felhasználó melyik opciót választotta.

10.6. Hírek

A híreket külön kategóriába lehetne sorolni (sport, rendezvény stb.), hogy ezzel is átláthatóbb legyen az oldal. Ezzel el lehetne kerülni azt, hogy a hírek a főoldalon ömlesztve jelenjenek meg, így javítva a felhasználói élményt. Szeretnénk még egy szűrési lehetőséget is, amivel hírekre lehetne keresni dátum vagy akar kulcsszó alapján.

10.7. Dokumentumtár

Iskolai sablonok (pl. kikérő, közösségi szolgálat igazolás) letöltése egy helyről, hogy a diákok könnyedén elérhessék a szükséges dokumentumokat várakoztatás nélkül.

10.8. Közösségi szolgálat követő

Egy felület lenne, ahol a diákok láthatják az IKSZ lehetőségeket, amikkel az iskola aktuálisan szerződésben áll, és jelentkezhetnének rájuk online.

10.9. Értesítéskezelés

A felhasználók be tudnák állítani, hogy miről kérnek értesítéseket. Például, ha valaki csak a chat üzenetekről szeretne, vagy a közelgő eseményekről.

10.10. QR-kód generátor

Minden eseményhez tartozhatna egy kód, amivel a helyszínen gyorsan be lehet csekkolni a portán vagy az adott esemény belépési pontján. minden felhasználó egy saját egyedi kódot kapna, ami egyszer használható fel csak,

10.11. „Digitális Szekrény”

Ez lényegében egy talált tárgyakra vonatkozó felület lenne, ahol, ha valaki talál egy feltételezhetően valaki számára fontos tárgyat azt fel tudja ide tölteni.

10.12. „Öreg Diák Fal”

Egy felület, ahol a már végzett diákok adhatnak tanácsokat a továbbtanulással kapcsolatban, vagy megoszthatják, hogy hova vették fel őket. Ez segíthetne a végzősöknek a pályaorientációban

10.13. Profil beállítások

Szeretnénk lehetőséget biztosítani a későbbiekben a felhasználóknak arra, hogy profil képet állíthassanak be, jelszót és email címet módosíthassanak, illetve felhasználónevet.

Összefoglalás

A DÖKNet azért készült, hogy megkönnyítsük a diákönkormányzat programjainak leszervezését. A programban több különböző jogosultság van, (`Diák`, `Tag`, `Elnök`, `Tanár`, `Admin`), amik kb. minden oldalrészen más dolgokat tudnak csinálni. A `Diák` felhasználók csak megtekinteni tudják az oldalt, a `Tag`-ok tudnak szavazni, valamint hozzáférnek a chathez. A `Tanár` és az `Elnök` felhasználók tudnak új eseményt létrehozni a naptárban, valamint szavazást és híreket is közzé tudnak tenni és ők is hozzáférnek a chathez. Az `Admin` ezek mellett még egy saját felülettel rendelkezik, ahol állítani tudja a jogosultságokat. A program minden oldalához tartozik egy adatbázis tábla, valamihez több is.

Az oldal főoldalán található egy képnézegető, valamint itt lehet megtekinteni a híreket is.

A naptárban lehet megtekinteni a közelgő eseményeket és azok leírásait. Található az oldalon egy galéria is, ahol a `Diák`-on és a `Tag`-on kívül minden más jogosultsággal rendelkező felhasználó tud képeket feltölteni az oldalra. A képek mindenki számára megtekinthetőek.

A hírszerkesztő felületén lehet híreket létrehozni. Több sablon is rendelkezésre áll (bejelentés, felhívás, kérdés), és ezek is megkönnyítik, hogy milyen hírt is olvas az olvasó.

A szavazás fülön a `Diák` és a `Tag` felhasználók nem tudnak szavazást létrehozni. Szavazást minimum két opcióval lehet létrehozni, minden szavazást ugyanazon az oldalon lehet megnézni. Az oldalon látszik, hogy jelenleg melyik szavazás aktív, és hogy melyik van lezárva.

A chat felületén a `Diák` felhasználókon kívül mindenki tud üzenetet küldeni. Az oldallal szeretnénk elősegíteni, hogy a diákönkormányzat sokkal könnyebben, és átláthatóbban tudja lebonyolítani az iskolában történő programokat.

Summary

The DÖKNet project is made for, because we would like to make the student council's work easily. In the program, there is various roles (`Student, Member, President, Teacher, Admin`) which are can do another things in almost every part of the sites. The `Student` users can just spectate the website, the `Members` can vote, and they have access to the chat. The `Teacher` and the `President` users can add new events in the calendar, and they can publish votes and news, and they too have access for the chat. In addition, the `Admin` users have an own panel, where they can manage the roles. Every part of the program have a database table, somewhere more than one.

In the main site of the program, there is a photo slider, and we can also view here the news.

In the calendar, we can view the upcoming events and its description. In the website, the visitors can find a gallery too, where users with any permission level other than `Student` or `Member` can upload photos to the site. The photos are available for everyone.

In the panel of the news editor can make the news. Many template are available to use (announcement, call, question), and it can help the reader what news is actually read.

In the vote window of the site, the `Student` and `Member` users are can't create votes. Only with minimum two options can create a vote, every vote are available to watch in the same site. In the site can view which votes are active, and which are is closed.

In the chat, with any permission level other than `Student` can write messages. With this website, we aim to help the student council organize school events more easily and transparently.

Ábrajegyzék

1. ábra: Visual Studio Code logó 5
2. ábra: XAMPP logó 5
3. ábra: Node.js logó 6
4. ábra: ChatGPT logó 6
5. ábra: Google Gemini 7
6. ábra: Next.js logó 8
7. ábra: React logó 8
8. ábra: TypeScript logó 9
9. ábra: MySQL logó 9
10. ábra: A főoldali képnézegető oldalt a logókkal 11
11. ábra: A főoldali hírek 11
12. ábra: Naptár 12
13. ábra: Galéria 13
14. ábra: Galéria képnézegetője 13
15. ábra: A Hír címe és az előre definiált borítóképek 14
16. ábra: A szövegszerkesztő mező 14
17. ábra: Kész hír eleje 15
18. ábra: Kész hír vége 16
19. ábra: Kész hír szerkesztése 16
20. ábra: Szerkesztés után 17
21. ábra: Kilistázott szavazások 18
22. ábra: Szavazatok létrehozása 19
23. ábra: Kész szavazás eleje 19
24. ábra: Kész szavazás opciói 20
25. ábra: Lezárt szavazás 20
26. ábra: Lezárt szavazás 21
27. ábra: Chat 22
28. ábra: Admin Panel 23
29. ábra: GlobalContext 24
30. ábra: Regisztráció 25
31. ábra: Bejelentkezés 26
32. ábra: A teljes adatbázis 27
33. ábra: users tábla 28
34. ábra: news tábla 29
35. ábra: reaction_types tábla 30
36. ábra: news_reactions tábla 31
37. ábra: comments tábla 32
38. ábra: chat_messages tábla 33
39. ábra: gallery tábla 34
40. ábra: events tábla 35
41. ábra: votes tábla 36
42. ábra: vote_options tábla 37
43. ábra: vote_votes tábla 38
44 ábra: DÖKNet logó 40
45 ábra: Háttérszín 40
46 ábra: Gombok színe 40

47 ábra: „mégse” gombok színe 41
48 ábra: A DÖKNet menüje 42
49. ábra: Hír INSERT 49
50. ábra: Szavazás lezárás UPDATE 49
51. ábra: bcrypt 50

Hivatkozások

https://www.prooktatas.hu/hirek/visual-studio-code-hasznalata

https://hu.wikipedia.org/wiki/XAMPP

https://nodejs.org/en/learn/getting-started/introduction-to-nodejs

https://chatgpt.com

https://hu.wikipedia.org/wiki/Gemini_(chatbot)

https://gemini.google.com/app

https://nextjs.org/docs#what-is-nextjs

https://en.wikipedia.org/wiki/React_(software)

https://en.wikipedia.org/wiki/TypeScript

https://www.oracle.com/mysql/what-is-mysql/